



Degree Project in Electrical Engineering and Computer Science

First cycle, 30 credits

Deep Reinforcement Learning on Social Environment Aware Navigation based on Maps

VICTOR SANCHEZ

Deep Reinforcement Learning on Social Environment Aware Navigation based on Maps

VICTOR SANCHEZ

Degree Programme in Electrical Engineering

Date: May 26, 2023

Supervisors: Josephine Sullivan (EECS - KTH), Chris Reinke (INRIA Grenoble),
Xavier Alameda-Pineda (INRIA Grenoble), Christophe Berenguer (Grenoble
INP)

Examiner: Jana Tumova (EECS - KTH)

Host organization: INRIA Grenoble - French Institute for Research in Computer
Science and Automation

Abstract

Reinforcement learning (RL) has seen a fast expansion in recent years of its successful application to a range of decision-making and complex control tasks. Moreover, deep learning offers RL the opportunity to enlarge its spectrum of complex fields. Social Robotics is a domain that involves challenges like Human-Robot Interaction which bears inspiration for development in deep RL. Autonomous systems demand a fast and efficient environment perception so as to guarantee safety. However, while being attentive to its surrounding, a robot needs to take decisions to navigate optimally and avoid potential obstacles. In this thesis, we investigate a deep RL method for mobile robot end-to-end navigation in a social environment. Using the observation collected in a simulation environment, a convolutional neural network is trained to predict an appropriate set of discrete angular and linear velocities for a robot based on its egocentric local occupancy grid map. We compare a random learning way to a curriculum learning approach to ameliorate speed convergence during training. We divide the main problem by analysing separately end-to-end navigation and obstacle avoidance in static and dynamic environments. For each problem, we propose an adaptation that aims to improve the surrounding awareness of the agent. The qualitative and quantitative evaluations of the investigated approach were performed in simulations. The results show that the end-to-end navigation map-based model is easy to set up and shows similar performance as a Model Predictive Control approach. However, we discern that obstacle avoidance is harder to translate to a deep RL framework. Despite this difficulty, using different RL methods and configurations will definitely help and bring ideas for improvement for future work.

Keywords

Deep Reinforcement Learning, Environment-aware navigation, Robotics, Artificial Intelligence

Sammanfattning

Förstärkande Inläring (RL) har sett en snabb expansion de senaste åren för sin fruktbara tillämpning på en rad beslutsfattande och komplexa kontrolluppgifter. Dessutom erbjuder djupinläring RL möjligheten att utöka sitt spektrum till komplexa områden. Social Robotics är en domän som involverar utmaningar som människa-robot interaktion som bär inspiration för utveckling i djup RL. Autonoma system kräver en snabb och effektiv miljöuppfattning för att garantera säkerheten. Men samtidigt som den är uppmärksam på sin omgivning, måste en robot fatta beslut för att navigera optimalt och undvika potentiella hinder. I detta examensarbete undersöker vi en djup RL-metod för mobil robot-end-to-end-navigering i en social miljö. Med hjälp av observationen som samlats in i en simuleringsmiljö tränas ett faltningsneuralt nätverk för att förutsäga en lämplig uppsättning diskreta vinkel- och linjärhastigheter för en robot baserat på dess egocentriska rutnätskarta över lokala beläggningar. Vi jämför ett slumpmässigt inläringssätt med läroplansinlärningsmetod för att förbättra hastighetskonvergensen. Vi delar upp huvudproblemet genom att separat analysera end-to-end-navigering och undvikande av hinder i statisk och dynamisk miljö. För varje problem föreslår vi en anpassning som syftar till att agenten bättre förstår sin omgivning. De kvalitativa och kvantitativa utvärderingarna av det undersökta tillvägagångssättet utfördes endast i simuleringar. Resultaten visar att den heltäckande navigationskartbaserade modellen är lätt att distribuera och visar liknande prestanda som en modell för prediktiv kontroll. Vi ser dock att undvikande av hinder är svårare att översätta till ett djupt RL-ramverk. Trots denna svårighet kommer användning av olika RL-metoder och konfiguration definitivt att hjälpa och ge idéer om förbättringar för framtida arbete.

Nyckelord

Djup Förstärkande Inläring, Människomedveten navigering, Robotik, Artificiell Intelligens

Résumé

L'apprentissage par renforcement (RL) a connu une expansion rapide ces dernières années pour ses applications à une gamme de tâches de prise de décision et de contrôle complexes. Le deep learning offre au RL la possibilité d'élargir son spectre à des domaines complexes. La robotique sociale est un domaine qui implique des défis tels que l'interaction homme-robot, source d'inspiration pour le développement en RL profond. Les systèmes autonomes exigent une perception rapide et efficace de l'environnement afin de garantir la sécurité. Cependant, tout en étant attentif à son environnement, un robot doit prendre des décisions pour naviguer de manière optimale et éviter les obstacles potentiels. Dans cette thèse, nous étudions une méthode de RL profond pour la navigation de bout à bout de robots mobiles dans un environnement social. À l'aide de l'observation recueillie dans un environnement de simulation, un réseau neuronal convolutif prédit un ensemble adapté de vitesses angulaires et linéaires discrètes pour un robot en fonction de sa carte de grille d'occupation locale égocentrique. Nous comparons une méthode d'apprentissage aléatoire à une approche d'apprentissage du curriculum pour accélérer la convergence durant l'entraînement. Nous divisons le problème principal en analysant séparément la navigation de bout à bout et l'évitement d'obstacles dans un environnement statique et dynamique. Pour chaque problème, nous proposons une adaptation visant à ce que l'agent comprenne mieux son environnement. Les évaluations qualitatives et quantitatives de l'approche étudiée ont été effectuées uniquement dans des simulations. Les résultats montrent que le modèle basé sur la carte de navigation de bout en bout est facile à déployer et affiche des performances similaires à celles d'une approche de contrôle prédictif de modèle. Cependant, nous discernons que l'évitement d'obstacles est plus difficile à traduire dans un cadre RL profond. Malgré cette difficulté, l'utilisation de différentes méthodes et configurations RL aidera certainement et apportera une idée d'amélioration pour les travaux futurs.

Mots-clés

Apprentissage par renforcement profond, Navigation consciente de l'humain, Intelligence Artificielle, Robotique

Research Center Presentation

The work realized for the thesis was done in the **French Institute for Research in Computer Science and Automation (INRIA)** center situated in the city of Grenoble. **INRIA** is composed of 215 project teams, most of which are shared with major research universities, more than 3,900 researchers and engineers explore new paths, often in an interdisciplinary manner and in collaboration with industrial partners to meet ambitious challenges.

I was part of the research team called RobotLearn based in the research center of Grenoble.



Acknowledgments

First, I would like to thank Dr. Chris Reinke for being a very kind and patient supervisor during my master's internship. I would also like to thank Dr. Xavi Alameda-Pineda for his goodwill.

In addition, I am thanking all the Ph.D. students, engineers, and assistants of the RobotLearn team for being supportive and helpful. Finally, I thank the French research lab center **INRIA** de l'université Grenoble Alpes for welcoming me.

Victor Sanchez

Contents

1	Introduction	2
1.1	Background	2
1.2	Problem	3
1.3	Executive Summary	3
1.4	Research Methodology	4
1.5	Structure of the thesis	4
1.6	Ethics and Societal Considerations	4
2	Background	6
2.1	Reinforcement Learning	6
2.1.1	Markov Decision Process	6
2.1.2	Value based Reinforcement Learning	7
2.1.3	Q-learning	8
2.1.4	Double Q learning	9
2.2	Deep Reinforcement Learning	9
2.2.1	Deep Q-Learning	10
2.2.2	Double Deep Q-Learning	10
2.2.3	Dueling Deep Q-learning	11
2.2.4	Other Deep Q-Learning	11
2.3	Deep Reinforcement Learning applied to Social Robotics	11
2.3.1	Social Robotics	11
2.3.2	Challenges	12
2.3.3	End-to-end navigation with Reinforcement Learning	13
2.3.4	Awareness of Environment in Reinforcement Learning	13
2.3.5	Contribution	14
2.4	Summary	14
3	Methodologies and Methods	15
3.1	Simulator	15

3.1.1	Multiparty Interaction Simulator	15
3.1.2	Reinforcement Learning Environment	16
3.2	Reinforcement Learning Key Objects	16
3.2.1	Observation Space	18
3.2.2	Action Space	19
3.2.3	Reward Signal	19
3.2.4	Simulation Parameters	20
3.2.5	Algorithms	21
3.2.6	Network Architectures	21
3.3	End-to-end Navigation	22
3.4	Social Environment Aware Navigation	23
3.4.1	Static Environment	23
3.4.2	Social Environment Aware Navigation	23
4	Results and Analysis	25
4.1	End-to-end Navigation - Results	25
4.1.1	Hyperparameter Tuning	25
4.1.2	Algorithm and Network	27
4.1.3	Policy Analysis	28
4.1.4	Comparison with MPC Approach	29
4.2	Static Environment Aware Navigation - Results	30
4.2.1	Reward Engineering Approach	31
4.2.2	Curriculum Learning Approach	33
4.2.3	Comparison with MPC Approach	34
4.3	Social Environment Aware Navigation - Results	36
4.3.1	Layered Input Approach	37
4.3.2	Framed Input Approach	38
4.3.3	Comparison with MPC Approach	39
5	Discussions and Conclusions	41
5.1	Discussions	41
5.2	Conclusions	42
5.3	Future work	42
	References	43
A	Additional Pseudo-Code	48

List of Figures

2.1	Agent and environment	7
3.1	Architecture of the MPI. The Simulation holds a list of Objects that represent physical entities such as walls, humans, and ARI. It also has a list of Scripts that define the behavior of the objects and a list of Processes such as the GUI.	16
3.2	Example of simulated room with robot, furniture, and humans	17
3.3	Flowchart of the interaction between the created gym-like environment, the Simulator, and the Agent	17
3.4	Global occupancy grid and observation space feature	18
3.5	Comparison of global occupancy grid with <i>No go zone</i> map	20
3.6	CNN-based Deep Q-Network	21
3.7	CNN-based Dueling Deep Q-Network	22
3.8	Multiple Layer Input	23
3.9	Multiple Layer Input CNN-based Deep Q-Network	24
3.10	Framed Input	24
3.11	Framed Input CNN-based Deep Q-Network	24
4.1	Example of simulated room with robot only	26
4.2	Reward per episode for several hyperparameter tuning - Plots obtained by averaging 10 repetitions - The shaded parts corresponds to the standard deviation.	26
4.3	Reward per episode for classic approach and dueling approach trained with Deep Q-Network (DQN) and Deep Double Q-Network (DDQN) algorithms - The Dueling DDQN is the approach that has a reduced standard deviation and that reaches the highest reward after 600 episodes - Plots obtained by averaging 10 repetitions - The shaded parts corresponds to the standard deviation.	28
4.4	Trajectories for end-to-end navigation performed by several RL agents . .	29

4.5	Trajectories for end-to-end navigation performed by an RL agent and an MPC Controller	30
4.6	Example of simulated room with robot and furniture in the center of the room	31
4.7	Performance of the agent trained in a static environment - Both approach reaches an averaged negative reward and the average number of collision is not decreasing - Plots obtained by averaging 10 repetitions - The shaded parts corresponds to the standard deviation.	32
4.8	Comparison of Global occupancy grid, <i>No go zone</i> occupancy grid with <i>warning zone</i> occupancy grid	32
4.9	Performance of the agent trained in a static environment with the improved reward signal - The agent reaches a reward comparable as the situation when the room is empty but the average number of collisions is more important - Plots obtained by averaging 10 repetitions - The shaded parts corresponds to the standard deviation.	33
4.10	Performance of the agent trained in a static environment with curriculum learning for No go zone (NGZ) reward-based and Warning zone (WZ) reward based - Both reward approaches are similar - Plots obtained by averaging 10 repetitions - The shaded parts correspond to the standard deviation.	34
4.11	Trajectories for end-to-end navigation and obstacle avoidance performed by a Reinforcement Learning (RL) agent and a Model Predictive Control (MPC) Controller for two configuration of rooms - The MPC Controller makes the robot avoid obstacles while the RL agent collide into the table and not maneuver away.	35
4.12	Example of trajectory for humans	36
4.13	Example of simulated room with robot and humans	37
4.14	Performance of dynamic obstacle avoidance with layered input approach - The agent's performance during training is affected by humans' density - Plots obtained by averaging 10 repetitions - The shaded parts correspond to the standard deviation.	38
4.15	Performance of dynamic obstacle avoidance for both layered input and framed input approach - The layered input version reaches a better reward and collides less on average than the framed input version - Plots obtained by averaging 10 repetitions - The shaded parts correspond to the standard deviation.	39

4.16 Trajectories for end-to-end navigation and human avoidance performed by a RL agent and a MPC Controller for two configurations of simulation (the background with the humans is just a screenshot to indicate human's area of evolution)	40
---	----

List of Tables

- 3.1 Discrete Action Space table 19
- 3.2 Reward weights 20
- 4.1 Chosen hyperparameters 27
- 4.2 Final Averaged Reward obtained over 10 repetitions 28
- 4.3 Percentage of the episode where the agent reached the goal obtained over 100 episodes and 10 repetitions 28
- 4.4 Quantitative performance of both approaches over 100 episodes. (An episode is considered as success if the agent reached the goal) 29
- 4.5 Quantitative performance of both approaches over 100 episodes. (An episode is considered a succeed if the agent reached the goal) 36

List of acronyms and abbreviations

AI	Artificial Intelligence
ANN	Artificial Neural Network
CL	Curriculum Learning
CNN	Convolutional Neural Network
DDQN	Deep Double Q-Network
DL	Deep Learning
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
GCRL	Goal-Conditioned Reinforcement Learning

HRI	Human-Robot Interaction
INRIA	French Institute for Research in Computer Science and Automation
MDP	Markov Decision Process
MPC	Model Predictive Control
MPI	Multiparty Interaction Simulator
RL	Reinforcement Learning
RNN	Recurrent Neural Network
SR	Social Robotics
TD	Temporal Difference

Chapter 1

Introduction

Human-Robot Interaction (HRI) is the study of how robots and humans can optimally interact with each other. It involves a range of sub-fields, such as artificial intelligence, robotics, and human-computer interaction. This chapter describes the context of the problem, the specific problem that is addressed, the goals of this project, and outlines the structure of the thesis.

1.1 Background

Within the **HRI** framework, **Social Robotics (SR)** is a field that focuses on the design of robots that can adapt to their environment and socially interact with humans. These robots may be used to perform tasks such as cleaning, navigation, and goal-oriented behaviors. For instance, the robot Roomba [1] from iRobot can vacuum and clean the floor in a social-friendly way which means avoiding humans or getting stuck. Human-like social robots also need to navigate in a social-friendly way by respecting the social distance between humans and static objects. They might also need to join a goal point to achieve a given task. These navigation needs can be solved by a control theory approach with for example a **MPC** based algorithm. They are designed to stabilize and optimize systems to guarantee an expected behavior in well-defined conditions. But such an algorithm may not be able to adapt to changing conditions or new situations. Also, they require a certain upfront design and engineering effort.

Artificial Intelligence (AI) based algorithms have more adaptability and flexibility and are less complex to engineer than the control theory-based approach. They are trained on large scaled data and can adapt under certain conditions to unknown situations. For example, **RL** approaches are based on trial error and are intuitive to design. **RL** offers robotics a

framework and set of tools for the design of these complex behaviors. Conversely, the challenges of robotic problems inspire developments in **RL**.

1.2 Problem

To navigate, robots should first use sensors such as cameras, lasers, or radar to gather information about their surroundings. Secondly, they need to be able to plan and execute safe and efficient paths through the environment, taking into account obstacles and other dynamic factors. Finally, they are required to perform in real-time, meaning that the robot must be able to quickly and accurately respond to changes in its environment. Navigating in a social environment is even more challenging for robots due to the need for interaction and understanding of human beings. Robots must navigate while preserving their integrity and that of surrounding humans. This thesis proposes to combine the problem of robot navigation with a social environment. The research question of the problem is the following: *"How efficient is Deep Reinforcement Learning (DRL) based method for navigation in a social environment?"*

1.3 Executive Summary

The main purpose of the thesis was to provide a **RL** based model that can perform an optimal path in a social environment while avoiding humans and static obstacles. As we used **AI** based algorithm, it was also the occasion to highlight some research challenges like learning an optimal path or avoiding objects in a social environment.

This project benefits to people who are interested in **RL** applications on robotics and **HRI**. Such technology of algorithms has to overcome social issues. The robot has respect for a social distance from humans and does not collide with them while going to a goal position. Also, it has not had to collide with static objects like furniture or walls.

The goal of this project is to investigate a **RL** based method in a social environment-aware navigation framework. This has been divided into the following two sub-goals:

1. Try to reproduce the combination of end-to-end navigation and obstacle avoidance
2. Analyze the challenges of a such problem through a different **RL** configuration
3. Compare the performance with a Model Predictive Control (MPC) approach

In this project, the final potential models were tested on simulation. It is assumed that the used simulation represents scenarios that are as real as possible. The thesis

demonstrates that **RL** based navigation is possible when obstacles are present just using an egocentric occupancy grid as input. Nevertheless, one must be careful with the design of the reward function as it has a huge impact on what the agent understands. Also, the **RL** navigation is much quicker but less efficient than the **MPC** approach due to the higher complexity of the **MPC**. From the work achieved, the thesis shows that **RL** based navigation is possible and can be learned to avoid people and other obstacles. Finally, we show that this is a promising approach, but there is a lot of room for improvement.

1.4 Research Methodology

During this work, we tuned several parameters. Some were specific to **RL** like the reward signal while others were more specific to **Deep Learning (DL)** like hyperparameters. To know if a learned behavior is social-friendly we need to collect and analyze numerical data like position and speed over time, which make the project also quantitative research. Thus, we can say that the thesis is both experimental and quantitative research.

1.5 Structure of the thesis

Chapter 2 introduces relevant background information about Deep Reinforcement Learning and Social Robot Navigation. Chapter 3 presents the methodology to solve the problem. Then, Chapter 4 contains the results obtained and their analysis. Finally, Chapter 5 exposes a conclusion of the thesis and some extensions that could be done in future work.

This thesis has been adapted for colorblind people and we used the color palette provided by Mr Wong [2].

1.6 Ethics and Societal Considerations

There are some ethical and social considerations when working with robots and **AI** [3]. First, autonomous robots must ensure the safety of their surroundings. Efficient sensors and robust algorithms are needed to avoid accidents. To further ensure safety, autonomous robots should be designed to allow human control that can bypass all other AI-based decisions and commands. Second, autonomous robots collect and store data such as images and voice recordings. It is thus important to protect individuals' privacy and ensure that data is used for legitimate purposes. Along the same lines, algorithms should be built to be transparent in their use of data and decision-making processes. It would certainly help to improve trust and acceptance, but it would also provide accountability

when something goes wrong. In addition, autonomous robots must be robust against cyberattacks. Important protections against unauthorized access must be implemented to ensure data integrity and confidentiality. Finally, the data and algorithms used to train autonomous robots should be as unbiased as possible. It is important to eliminate bias during the design stage and to regularly monitor and test robot performance to ensure fairness and equality.

Chapter 2

Background

This chapter provides background information **RL**, **DL**, and **Deep Reinforcement Learning (DRL)**. Additionally, this chapter introduces Social Robotics (**SR**), Navigation, and **DRL** applied to robotics.

2.1 Reinforcement Learning

RL [4] is a machine-learning framework that trains an agent to make decisions in an environment. In a **RL** problem, the agent interacts with the environment by taking actions and receives rewards or punishments for their actions. Its goal is to learn a policy that maximizes the cumulative reward through time.

2.1.1 Markov Decision Process

The **Markov Decision Process (MDP)** is a mathematical framework for modeling decision situations in which outcomes are affected by a series of decisions and their random events. **MDP** is characterized by the fact that the probability of transitioning to a new state depends only on the current state and the action taken. It defines a four-tuple (S, A, P, R) , S is the set of states called *state space*, A is the set of actions called *action space*, P is the probability that action a_t in state s_t will lead to state s_{t+1} : $P = Pr(s_{t+1}|s_t, a_t)$. It is called usually the *transition probability*. R is the immediate reward received after switch from s_t to s_{t+1} while taking a_t .

RL uses **MDPs** where the reward or the transition probability is unknown. A simulator is typically restarted many times from a uniformly random initial state providing implicit transition probabilities. Inspired by the way animals learn by trial and error, it has been successfully used in a variety of applications including game playing [5] or robot control [6].

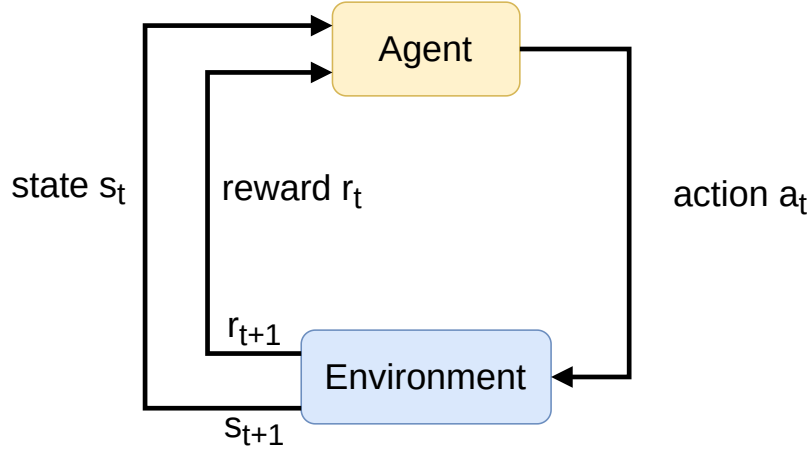


Figure 2.1: Agent and environment

In each step, the agent takes an action a_t respective to its policy $\pi(a_t|s_t)$, where s_t is the current observation from the environment. The agent receives a reward r_{t+1} and the next observation s_{t+1} from the environment. The goal is to improve the policy to find the one that maximizes the sum of rewards.

2.1.2 Value based Reinforcement Learning

There are two main types of **RL**: value-based and policy-based. In value-based **RL**, an agent learns a function that predicts the expected reward for each state or action. In policy-based **RL**, agents directly learn policies that assign actions to states. Some algorithm called Actor-Critic combines both techniques [7].

The state value function V^π maps the current state to the expected discounted reward collected following a policy π when starting at this state.

$$V^\pi(s) := E\left[\sum_{t=1}^{\infty} \gamma^{t-1} r(s^\pi, a^\pi) | s_1^\pi = s\right] \quad (2.1)$$

We call the parameter γ the discount factor and r the reward function mapping the immediate reward received from the environment. The state-action value function of a

policy π is called the Q-function and is denoted by: $Q^\pi(s, a)$. It maps an initial state s and action a to the collected reward assuming that after this first action, the decisions are driven by π . It is defined as :

$$Q^\pi(s, a) := r(s, a) + E\left[\sum_2^\infty \gamma^{t-1} r(s_t^\pi, a_t^\pi) | s_1^\pi = s\right] \quad (2.2)$$

The state value function and the state action value function are linked by the following relation : $V^\pi(s) = \sum_a \pi(a | s) Q^\pi(s, a)$.

The Q function is the value of an optimal policy when the first selected action is imposed. The Bellman equation for the Q-function is then :

$$Q^\pi(s_t, a_t) = E[r(s_t, a_t) + \gamma E[Q^\pi(s_{t+1}, a_{t+1})]] \quad (2.3)$$

2.1.3 Q-learning

The algorithm to learn the optimal Q function is called the Q-learning algorithm (algo. 1).

Algorithm 1 Q-Learning

Input: policy π , integer $nb_episodes > 0$, learning rate α , discount factor γ , exploration rate ϵ

Output: value function Q ($\approx q_\pi$ if $nb_episodes$ is large)

```

1 Initialize  $Q$  arbitrarily (e.g.,  $Q(s, a) = 0$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$ , and  $Q(\text{terminal-state}, \cdot) = 0$ )
   for  $i \leftarrow 1$  to  $nb\_episodes$  do
2     Observe  $s_0$ 
        $t \leftarrow 0$ 
       repeat
3         Choose action  $a_t$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
           Take action  $a_t$  and observe  $r_{t+1}, s_{t+1}$ 
            $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_b Q(s_{t+1}, b) - Q(s_t, a_t))$ 
            $t \leftarrow t + 1$ 
4       until  $s_t$  is terminal;
5 return  $Q$ 

```

The $y_t = r_{t+1} + \gamma \max_b Q(s_{t+1}, b)$ is the **Temporal Difference (TD)** target, and $y_t - Q(s_t, a_t)$ is the **TD** error.

The optimal Q-function Q^* obeys the following Bellman optimality equation:

$$Q^*(s_t, a_t) = E[r + \gamma \max_b Q^*(s_{t+1}, b)] \quad (2.4)$$

It means that the maximum return from a state action pair (s, a) is the sum of the immediate reward r and the return discounted by γ obtained by following the optimal policy until the end of the episode (i.e., the maximum reward from the next state s'). The expectation is computed both over the distribution of immediate rewards r and possible next states s' . We can represent the Q-function as a table containing values for each combination of s and a .

The ϵ parameter is introduced respectively to a method called ϵ -greedy. It means that the agent takes random action with a probability ϵ . This parameter can be either static or dynamic during the learning phase. One commonly used case is to fix it to one at the beginning and to linearly decrease it until it goes to a minimum with the rule $\epsilon_t = \max(\epsilon_{t-1} * \epsilon_{decay}, \epsilon_{end})$ and $\epsilon_0 = \epsilon_{start}$.

2.1.4 Double Q learning

In standard Q-learning, the max operator is used two times. One time for selection (when the agent acts) and one time for evaluation (where we do the update). This makes it more likely to select overestimated values, resulting in over-optimistic value estimates. The Double Q-learning [8] algorithm decouples the selection from the evaluation by implementing a Q-function for each of them.

2.2 Deep Reinforcement Learning

DRL combines the principles of **RL** with the power of **DL**. This involves training an **Artificial Neural Network (ANN)**, known as a deep neural network, to make a series of decisions in the environment to maximize the reward signal. We use **ANN** to approximate the Q-function and to be able to handle continuous state and action spaces. One of the main advantages of **DRL** is its ability to learn from raw data and make decisions without the need for manual feature engineering. This makes it particularly suitable for tasks such as playing games and controlling robots in complex environments. Classical **RL** relies on hand-designed features to represent the state of the environment and the actions that can be taken. **DRL** can learn directly from the raw data and can handle higher dimensional state space.

2.2.1 Deep Q-Learning

In a domain like Robotics, it is impractical to represent the Q -function as a table. Instead, we train a function approximator, such as a neural network with parameters θ , to estimate the Q -function, i.e. $Q(s_t, a_t; \theta) \approx Q^*(s_t, a_t)$. We use a **DL** equivalent to the Q-learning algorithm called the **DQN** algorithm. The loss to minimize at step i and time step t is :

$$L_i(\theta_i) = E_{s_t, a_t} [(y_i - Q(s_t, a_t; \theta_i))^2] \quad (2.5)$$

where $y_i = r_i + \gamma \max_{a'} Q(s', a'; \theta_{i-1})$ is the target.

It exists several types of network models and architectures depending on the information we have on the environment. In a simple case like chess, the action space is discrete, and an algorithm like the **DQN** algorithm can be efficient [9]. In approaches like Social Robotics, where we can define the action space as discrete and the state space as continuous a **DQN** can also be used. Depending on the input data we have and the final behavior we want, the shape of the neural network can have a different form. For example, if we want the agent to use real-time data to make decisions, we should use a **Recurrent Neural Network (RNN)**. Also, in the case where the input is an image a **Convolutional Neural Network (CNN)** followed by a Fully Connected Network [10] seems adapted. Knowing that a **CNN** are specialized networks for image processing and that **RNN** are specialized networks for time-dependent data.

2.2.2 Double Deep Q-Learning

The equivalent of Double Q-learning also exists when dealing with deep learning. We use a "Policy Network" for evaluation and a "Target Network" for selection. The loss to minimize at each step i then become :

$$L_i(\theta_{policy}) = E_{s,a} [(r_i + \gamma \max_{a'} Q(s', a'; \theta_{target}) - Q(s, a; \theta_{policy}))^2] \quad (2.6)$$

The update of the Target Network can be done in a periodic way by setting its parameter as equal to the Policy Network. It also can be done in a soft way by making updating the weights of the target network ϕ at each time step by multiplying the weights of the Policy Network θ by a softness parameter τ : $\phi = \theta \cdot \tau + \phi \cdot (1 - \tau)$.

2.2.3 Dueling Deep Q-learning

From now every approach proposes to estimate the Q-value which depends on both action and state. Another perspective called Dueling DQN [11] divided the Q-value estimation into two parts: The value function $V(s)$ and the Advantage function $Adv(s, a)$. The value function estimates how good it is to be in state s and the advantage function estimates the advantage of taking action in that state. Knowing that the advantage function [12] was defined as $Adv(s, a) = Q(s, a) - V(s)$ it is easy to fuse V and Adv to represent the final Q-values. This approach proves that state values can be learned separately from the effect of action advantages. It helps to identify states where actions have no impact.

2.2.4 Other Deep Q-Learning

A more complete approach considers the combination [13] of the DQN variants to build a more robust learning agent. Several improvements to DQN can be successfully integrated into a single learning algorithm that achieves state-of-the-art performance.

Finally, there are parameters that are proper to DL which are called hyperparameters. For example, the learning rate defines the step size for each iteration while moving into a minimum loss function. A batch is a sample of data we use to input in a neural network.

2.3 Deep Reinforcement Learning applied to Social Robotics

In the context of robotics, deep reinforcement learning can enable robots to learn several tasks and behaviors without the need for explicit programming. This includes tasks such as navigating unfamiliar environments, grabbing and manipulating objects, and interacting with people.

2.3.1 Social Robotics

Social robotics [14] is a field of robotics focused on the design and development of robots that can socially interact with humans. Designing robots that can understand and respond appropriately to human standards is one of the biggest challenges in SR. It requires several technologies such as natural language processing [15], facial recognition [16], and gesture recognition. Social robots have potential applications in healthcare [17], education [18], and entertainment [19]. In healthcare, social robots can be used to assist with tasks such as

medication reminders and social interaction. Social robotics is a field that has the potential to improve the way we live and work and could enhance our social interactions.

2.3.2 Challenges

This section introduces the challenges and previous work related to the project.

Navigation

The use of two-dimensional maps to navigate was used for localization in a 3D maze environment [20]. Here the navigation is done with a map and is decomposed into three tasks: First, observing the 3D environment and comparison to the 2D map to guess the position. They also provide a solution to learn how to follow a plan to reach a goal point. The network used is a CNNs to read to visual input and build the visible local map. The approach is using an advanced policy-based method: A3C [21].

Reward Engineering

Reward Engineering is a DRL research field that involves challenges especially in domains like Social robotics [22]. To shape the reward function, we need to understand the task and the desired behavior of the agent. This requires domain-specific expertise that may not be available. Also, reward shaping can change the distribution of the data the agent sees, which can make learning more unstable and harder to converge.

In other cases, it can be interesting to provide a reward that is sparse. For example, we give no reward to the agent during the whole episode and we give a positive reward only when the goal is achieved. Having a sparse reward can dangerously increase the training time procedure because the agent does not directly understand the real goal. Some approaches allow sample-efficient learning from rewards that are sparse and therefore avoid the need for complicated reward engineering [23]. The opposite of having a sparse reward would be having a frequent reward signal.

Learning Performance Improvement

In classic RL, we aim to induce a unique optimal solution for a specific task. But in a field like robotics, we might have several goals to achieve in a single task. The decision-making is thus according to different goals. Goal-Conditioned Reinforcement Learning (GCRL) [24] gives a solution for reward tuning to multiple goal tasks that can be applied in a case where a robot should reach multiple goal points. The agent must either learn to perform multiple tasks simultaneously or decompose the long-term goal into intermediate

sub-goals while learning to reach them with a unified policy.

Another decomposition of sub-tasks can be by the level of difficulty from simple to hard. **Curriculum Learning (CL)** [25] approach proposes to design graduated curriculums to improve learning performance. It has been shown [26] that it can significantly ameliorate speed convergence and avoid some local minima.

2.3.3 End-to-end navigation with Reinforcement Learning

Some research work has been done aiming to make an agent navigate in a dynamic environment. A work about end-to-end driving applied to autonomous vehicles, [27] tackled the problem of reward engineering by creating a sparse reward function to improve exploration. To overcome the sparsity of the reward, they used a Goal-Conditioned approach using only navigation view maps. End-to-end navigation has also been applied to robotics with 2D Maps [28] and showed robustness to sensor noise. Here they compared the performance of a simple Dueling DQN [11] with Prioritized Experience Replay [29] with the Proximal Policy Optimization algorithm. An improvement of this end-to-end navigation has also been done by the same researchers [30] on multi-robot collision avoidance.

2.3.4 Awareness of Environment in Reinforcement Learning

In Robotics, the awareness of the environment is different depending on what is around the robot. For example, if two robots are around each other, the robots will behave without taking into account any special social conditions. But if a robot is navigating around humans, the robot must behave in a social-friendly way. Previous work [31] studied the social aspect of how robots should approach humans. They provided a framework that enables social robots to safely and socially approach a human.

In **RL**, the behavior of the agent is ruled by the reward signal. Former work [30] proposed a map based-model to avoid collision with other robots while dealing with a multi-agent system. Their reward has a component to make the agent avoid obstacles, whether they are static or dynamic. Another work [32] proposes also a map-based model that enables a robot to navigate around pedestrians. Their reward takes into account static obstacles and humans. The closer the robot is to a pedestrian, the lower the reward is. For both approaches, the model is map based and the reward is almost the same. It means that the **RL** approach does not consider the type of dynamic obstacle. The agent is trained similarly to avoid moving obstacles whether they are humans or not. Of course, there exists also a research field that focuses on what the robot must not do when humans are around it. For example, the work by Chen et al. [33] gives insight into how robots should behave when

crossing humans. This kind of approach is not considered in the thesis. This thesis will follow the works that make no difference between humans and other moving obstacles. The term "social" refers to the fact that the agent learns to navigate in an environment where there are humans. Nevertheless, the developed model does not behave differently for humans and other potential moving obstacles.

2.3.5 Contribution

This work recreates the use of the **DRL** methods previously exposed to solve an end-to-end navigation task while avoiding obstacles. Obstacle avoidance learning was studied both for static and dynamic obstacles. To take into account obstacles 2D maps of surrounding objects and entities were used. Finally, this work proposes to make a comparison with a control theory-based algorithm.

2.4 Summary

To conclude we saw that the problem of robot navigation in a social environment solved with a **DRL** approach has some important challenges to overcome. First obstacle avoidance, social-aware navigation, and the fact that the robot should have a socially friendly path. Secondly, there is a technological challenge that concerns more **DRL** itself with the engineering of the reward signal. Its impact on providing an efficient path planning understanding for the agent will be challenging to master and will need to be engineered.

Chapter 3

Methodologies and Methods

As described in section 1.4, the research will be both experimental and quantitative. The main problem of navigation in a social environment involves multiple sub-challenges that were presented before. We will use different techniques to solve this problem. We decided to structure our work into two problems as follows:

- End-to-end navigation
- Social environment aware navigation

This chapter presents first the simulator in section 3.1.1 and the RL environment used in section 3.2. Then it presents the approach and the methodology used to solve each sub-problems.

3.1 Simulator

3.1.1 Multiparty Interaction Simulator

The **Multiparty Interaction Simulator (MPI)** simulator was developed by members of the **INRIA** team and was used during the thesis. It has a graphical user interface (GUI) as a method to visualize it. The interface is based on the PyGame [34] framework. The GUI functions allow for visualizing of multiple properties. We will mainly use it for the PyBox2D 3.2 bodies of all Objects including robots, human agents, and objects such as walls or furniture.

The simulation is a top-level interface for a simulator that includes various elements such as Objects, Components, Scripts, and Processes. It allows for the execution and resetting of simulations, which occur in discrete steps and involves the calling of the "step" method for each element in a specific order. The simulation also includes a physical

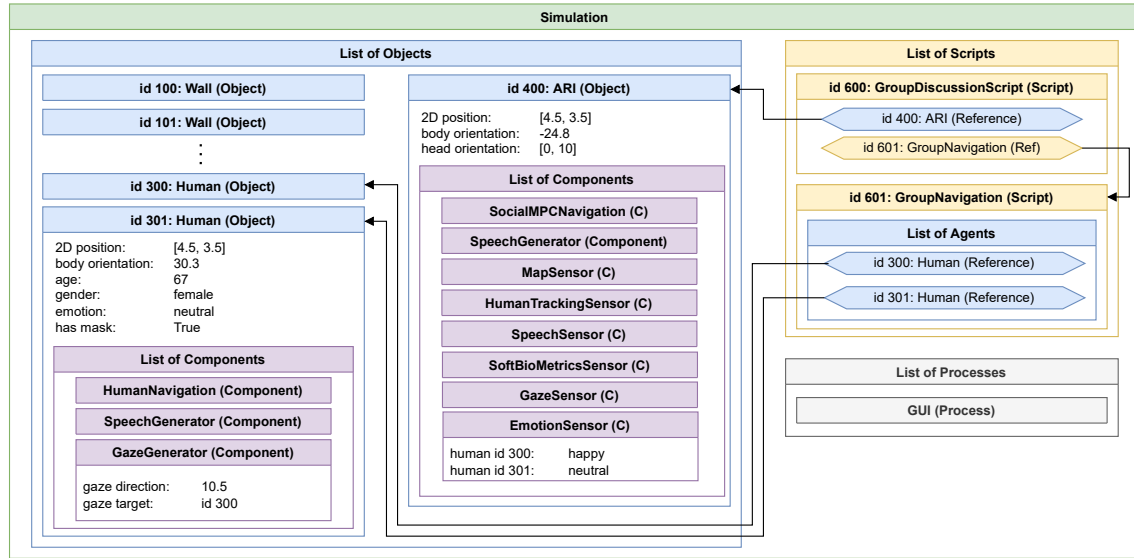


Figure 3.1: Architecture of the MPI. The Simulation holds a list of Objects that represent physical entities such as walls, humans, and ARI. It also has a list of Scripts that define the behavior of the objects and a list of Processes such as the GUI.

simulation of 2D bodies using the PyBox2D [35] framework. Elements such as objects and their components are responsible for generating behaviors and sensing properties of other objects.

3.1.2 Reinforcement Learning Environment

Using the glsMPI simulation, we implemented an environment structured as the environment developed by OpenAI [36] available in the python library gym. There is first a *reset* function that initializes the simulation and the robot's position. It eventually adds objects and entities. It returns an observation of that initialized environment that will be used by the **RL** agent. Then there is a *step* function that takes an action from the agent. It steps the simulation and the position of each object and entity. This function collects data from the simulator like the position and velocities of humans and provides it to the agent.

3.2 Reinforcement Learning Key Objects

In the following, we describe the key objects of the reinforcement learning framework, including the observation and action space, the reward signal, the simulation parameters, the algorithms, and the network architecture.

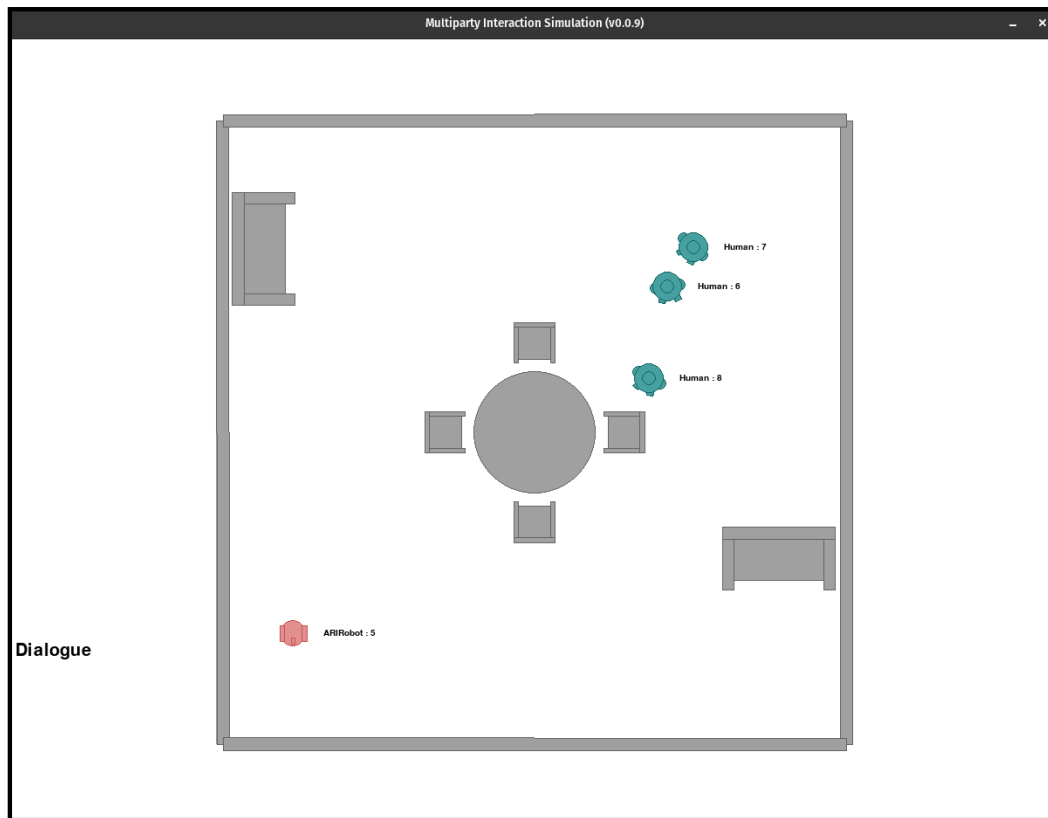


Figure 3.2: Example of simulated room with robot, furniture, and humans

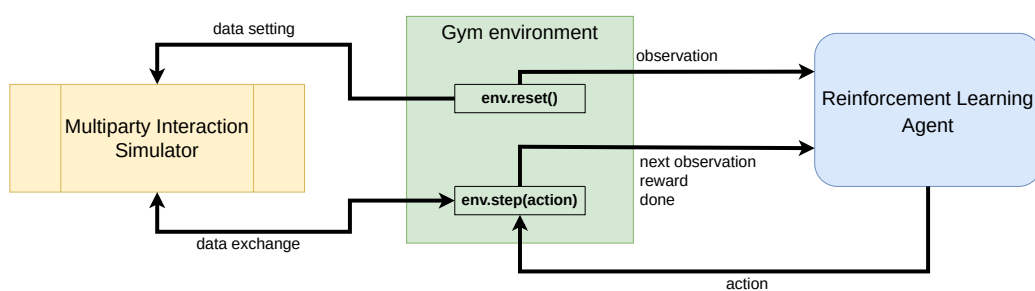


Figure 3.3: Flowchart of the interaction between the created gym-like environment, the Simulator, and the Agent

3.2.1 Observation Space

Lots of gIsRL literature uses the names "state space" and "observation space" to define the same thing. We consider as "state space" the whole state of the simulation. The observation space contains only some information about it. In this thesis, the name "observation space" will be employed as we consider an "observation" of a state and not the state itself. The **MPI** can produce an occupancy grid (fig. 3.4) containing both static (furniture or walls) and dynamic entities (humans). It also contains several functions that can provide measurements like distance or angle.

We will also use what is called *occupancy grids* which are grids that contain one when there is an object or entity and where there is nothing. It gives a simple binary encoding that is intuitive and simple to handle. From the global occupancy grid, it is possible to obtain an oriented local egocentric grid of the objects and entities surrounding the robot. The observation space is composed first of the local occupancy grid (64x64). Then there is information concerning the goal position, such as the distance between the robot and the goal and their relative angle. For each sub-problem, the observation space will be slightly different and will be detailed afterward.

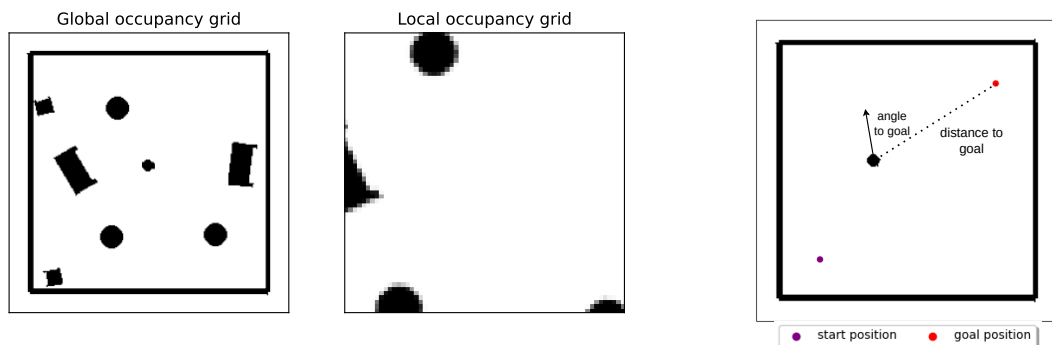


Figure 3.4: Global occupancy grid and observation space feature

Moving 2D maps as input

As we have seen in chapter 2 it is common to use an image as input for **DRL**. The images used for Atari Game [5] were screenshots of the game, each of them being similarly structured. Then the agent can focus on certain zones of these images, creating what is called the salient region. Salient regions are the regions where the neural network focuses on making its decisions. So, when the image has the same overall structure, the salient

region is easy to determine for the **CNN**. Here we take as input an egocentric occupancy grid. This grid follows the robot's movement and its content changes whenever the robot moves. So, in every simulation step, a different image is given to **CNN**. Thus, **CNN** analyzes the salient regions to know where to focus. It also analyzes the patterns of the potential surrounding objects to know what to focus on. These additional parameters of moving input maps have an impact on learning and make the challenges harder to master.

3.2.2 Action Space

The action space of the agent is a set of linear and angular velocities in discrete space. Action corresponds to a linear velocity V and angular velocity Ω . The simulation clips the linear velocity in $[-0.1 \text{ m/s}, 5 \text{ m/s}]$ and the angular velocity in $[-1.57 \text{ rad/s}, 1.57 \text{ rad/s}]$. The following table describes the permissible action and their corresponding velocities.

description	stop	go straight	curve right	curve left	turn right	turn left
action	0	1	2	3	4	5
linear velocity	0	1	0.8	0.8	0	0
angular velocity	0	0	-1	1	-1	1

Table 3.1: Discrete Action Space table

3.2.3 Reward Signal

The agent's goal is to maximize the cumulative reward and the reward function specifies what the agent must achieve.

$$r = r_t + r_{improvement} + r_{dynamic} + r_{static} + r_{goal} \quad (3.1)$$

First, the agent receives a reward per step: $r_t = -\omega_t$ to minimize the number of steps to join the goal position. To give the agent an idea of improvement we provide a reward that is the difference between the former distance to the goal and the actual distance to the goal :

$$r_{improvement} = \omega_{improvement} \cdot [d_{(t-1)}(agent/goal) - d_{(t)}(agent/goal)] \quad (3.2)$$

Knowing that $d_{(t)}(agent/goal) = \sqrt{(x_{agent} - x_{goal})^2 + (y_{agent} - y_{goal})^2}$.

Then, if the agent gets closer to the goal it receives a positive reward, and if the agent goes away from it receives a negative reward.

The next three rewards are final rewards which mean that the agent receives these rewards when the episode terminates. An episode ends if the agent has reached the goal, then it

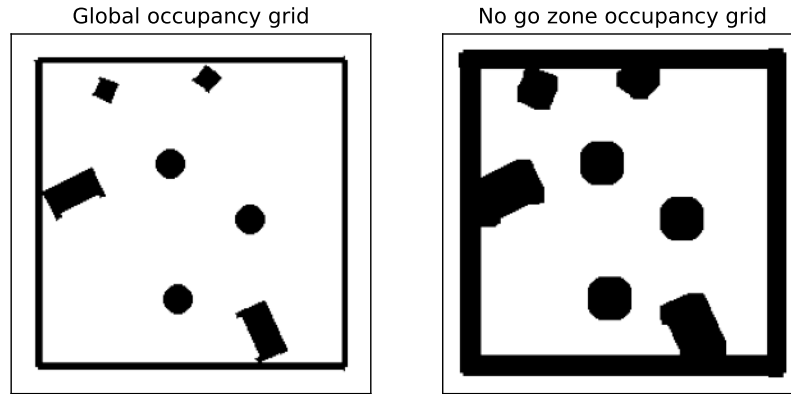
receives $r_{goal} = \omega_{goal}$ or when the agent collides into furniture (resp. a human) it receives $r_{static} = -\omega_{static}$ (resp. $r_{dynamic} = -\omega_{dynamic}$). We will take the following parameter :

Reward Weight	ω_t	$\omega_{improvement}$	ω_{goal}	ω_{static}	$\omega_{dynamic}$
Values	5	200	500	500	500

Table 3.2: Reward weights

Some weights values like ω_{static} , ω_{goal} or $\omega_{dynamic}$ were choose following prior work. Other weights like $\omega_{improvement}$ and ω_t were tuned to obtain the behavior.

To detect whether the agent collides with the furniture we define a zone of 0.5m around each object called *No go zone*. It creates then a map of *No go zones* (fig. 3.5) which defines the forbidden position. In case the agent is in one of these positions, the episode ends, and the agent receives the reward r_{static} .

Figure 3.5: Comparison of global occupancy grid with *No go zone* map

3.2.4 Simulation Parameters

A simulation can contain a given number of furniture, agents, and humans. For each episode, their initial position can be defined as random or fixed.

Several biases will have to be considered and avoided. First, the start and goal position of the agent is fixed for each episode, it will only learn an optimal trajectory for this configuration. But as the goal of the project is to build a model that can perform end-to-end navigation, the agent would have to be trained with both random start and goal

position to be able to achieve the requirements.

Also, an important bias could come from the observation the agent gets. If the agent has the global occupancy grid and if furniture has a fixed position over every episode, the agent will only learn their configuration and will not potentially learn to avoid static objects wherever they are. Nevertheless, if the agent has a local map, then it can learn to generalize and learn what to not go forward when there is furniture in front of it, for instance. A trade-off will have to be considered between the number of furniture and their randomness. With dynamic entities, the challenge is even more complex compared to a static object. As their trajectory is sort of unpredictable, the agent will have to learn to avoid them efficiently.

3.2.5 Algorithms

Our **RL** environment contains both a discrete action space and an observation space. To estimate the Q-function we will use the algorithms which were presented in the section 2.2 : Deep Q-Network and Double Deep Q-Network. A comparison of performance will be presented in the following parts.

3.2.6 Network Architectures

To estimate the Q-value (eq.2.4) we will consider first a network (fig. 3.6) which is composed of a **CNN** for the treatment of the input map. The input about goal position is merged with the flattened output of the **CNN** and finally input in a multi-layer perception with two hidden layers.

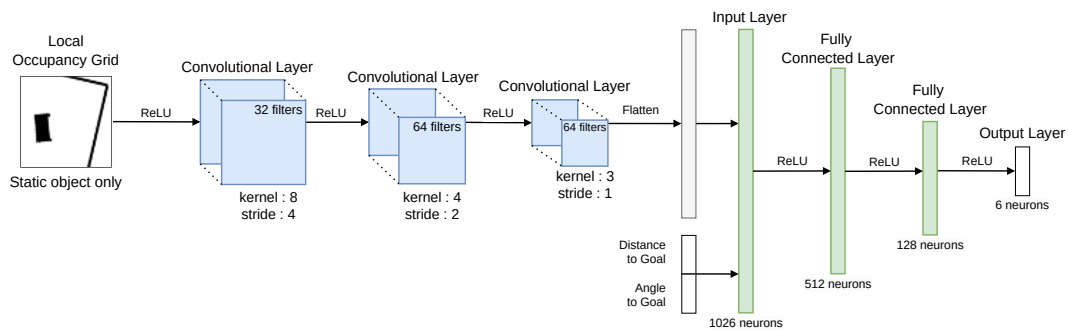


Figure 3.6: CNN-based Deep Q-Network

We will also consider a Dueling DQN approach the network (fig. 3.7) is based on the

same front-end perception module but estimates both the value function and the advantage function that are finally fused to represent the final Q-values (eq.2.4).

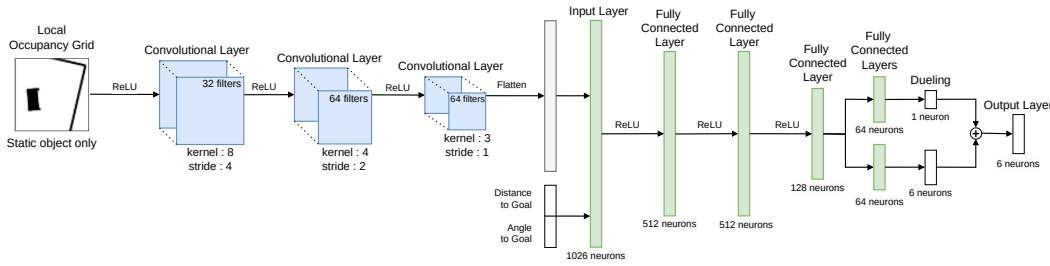


Figure 3.7: CNN-based Dueling Deep Q-Network

These network architectures are inspired by two main previous works. First, the one that uses CNN to solve Atari Game with DRL [5]. Second, the one that use the same approach [37] but in an end-to-end navigation framework.

3.3 End-to-end Navigation

In this first sub-problem, the goal is to highlight the challenge of end-to-end navigation and to obtain a trained agent that can perform end-to-end navigation. The environment where the agent will be trained is going to be an empty squared room with four walls. Its starting and goal positions will be random for each episode.

The observation space is an egocentric map of the robot. Also, it contains information about the goal position, like the distance to the goal and the angle to the goal. The network architecture will be the same as presented before (section 3.2.6). We choose to show some experiments of tuning that we think are relevant to understand the challenge of end-to-end navigation. First, we did a hyperparameter tuning (section 4.1.1) on some parameters that are known to be important, like the learning rate of the epsilon decay for the epsilon greedy policy. Then we proceeded to an algorithm comparison between DQN and Double DQN (section 4.1.2). But also to a network comparison between our CNN-based Deep Q-Network (fig. 3.6) and its Dueling variant (fig. 3.7). In addition, we will analyze different policies obtained with the trained agent. Finally, we will compare the performance of the model obtained with the approach of the MPC 4.1.4.

3.4 Social Environment Aware Navigation

The social environment-aware navigation will be divided into two main parts. First, an approach that focuses on navigation in a static environment. Then an approach that analyses navigation in a dynamic environment.

3.4.1 Static Environment

In this first sub-problem, the main challenge is to train a model to perform end-to-end navigation in a static environment. We will first present an approach based on reward engineering. (section 4.2.1) Then we will focus on a **CL** based approach to pay attention to the impact on the learning process. (section 4.2.2)

Finally we will analyze the different policies obtained and compare them to the Model Predictive Control approach (section 4.2.3).

3.4.2 Social Environment Aware Navigation

In this second sub-problem, the main challenge is to train a model for social-aware end-to-end navigation.

We used two forms of observation space here. The first approach (section 4.3.1.) is a multiple-layer egocentric occupancy grid of the robot (fig. 3.8). The first layer will be the occupancy grid of the surrounding static objects. The next layers will contain the position, orientation, and velocity of the surrounding humans. Also, it will contain information about the goal position, like the distance to the goal and the angle to the goal. The network architecture (fig. 3.9) will be like the former approach.

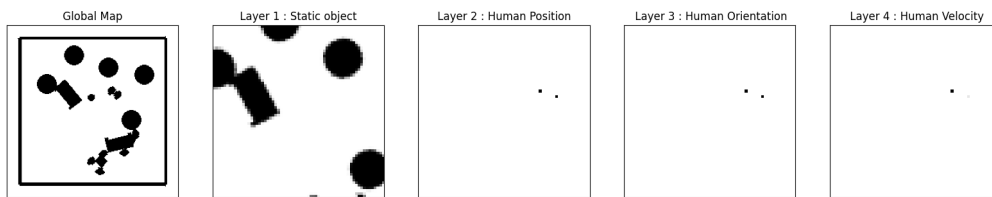


Figure 3.8: Multiple Layer Input

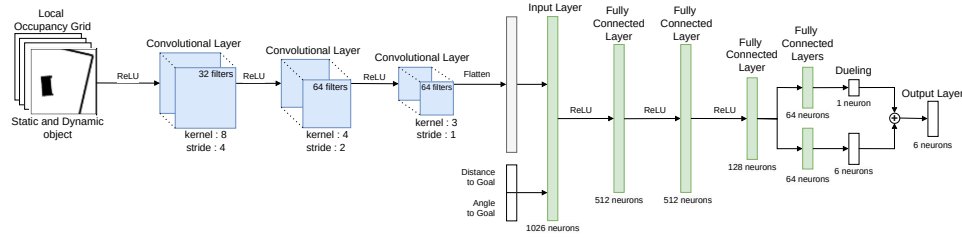


Figure 3.9: Multiple Layer Input CNN-based Deep Q-Network

For a second approach, (section 4.3.2.) is a framed layer egocentric occupancy grid of the robot (fig. 3.10). A given layer will contain the position of the static objects and the dynamic entities and a given time step. (fig. 3.11)

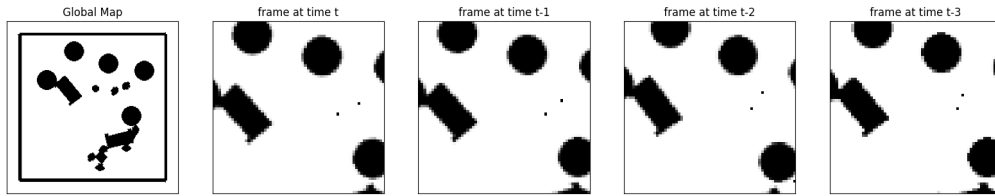


Figure 3.10: Framed Input

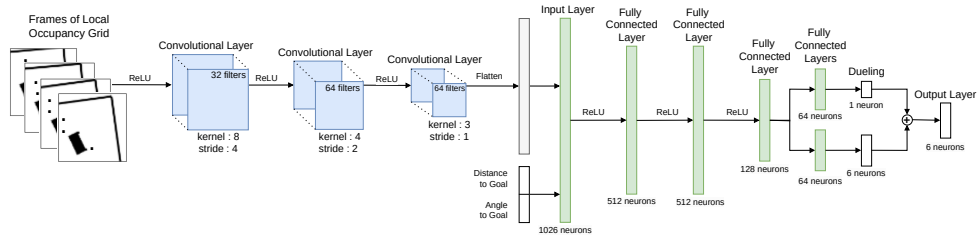


Figure 3.11: Framed Input CNN-based Deep Q-Network

The difference between the two networks is that the dynamic information is given differently. Given that we aimed to see if one version was more appropriate and better than the other. Finally we will proceed to a comparison performance with the MPC approach (section 4.3.3).

Chapter 4

Results and Analysis

In this chapter, we present the results obtained and analyze them. We also highlight what is challenging for each problem and how it was or could be overcome. This chapter follows the structure of the two main problems presented in the former chapter.

4.1 End-to-end Navigation - Results

As we announced before, the state space is composed of the egocentric local occupancy grid but also of the distance to the goal and the angle to the goal. The neural networks used were the ones presented in fig. 3.6 and fig. 3.7 and the algorithm is the **DDQN**.

The aim is to perform end-to-end navigation and the environment where the agent is trained in an empty room with four walls (fig. 4.1). The agent has a random starting and goal position, which is different for every episode. Every experiment was launched at 10 repetitions to obtain intelligible results. We then averaged each result over these 10 repetitions, which gave a mean (line) and a standard deviation (shaded part) for each data that we looked at. Finally, the maximum number of steps for an episode is 300 and the total number of episodes is 600.

4.1.1 Hyperparameter Tuning

As a first approach, we decide to tune the hyperparameter to make sure that the neural network learns in an appropriate and optimized way. We evaluate the relevance of each hyperparameter by plotting the reward per episode (fig. 4.2). The optimizer used is the Adam optimizer.

First, regarding the results of the batch size, it has a weak influence on the learning process. Considering the training time and computational resources into account, we kept a batch size of 32. The learning rate, on the other hand, has an important influence. With a

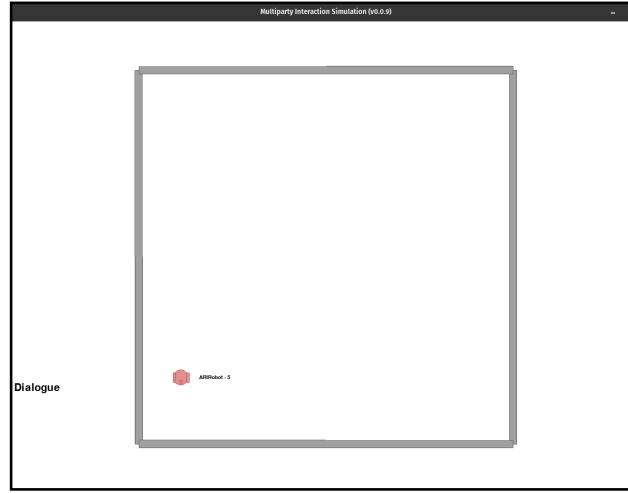


Figure 4.1: Example of simulated room with robot only

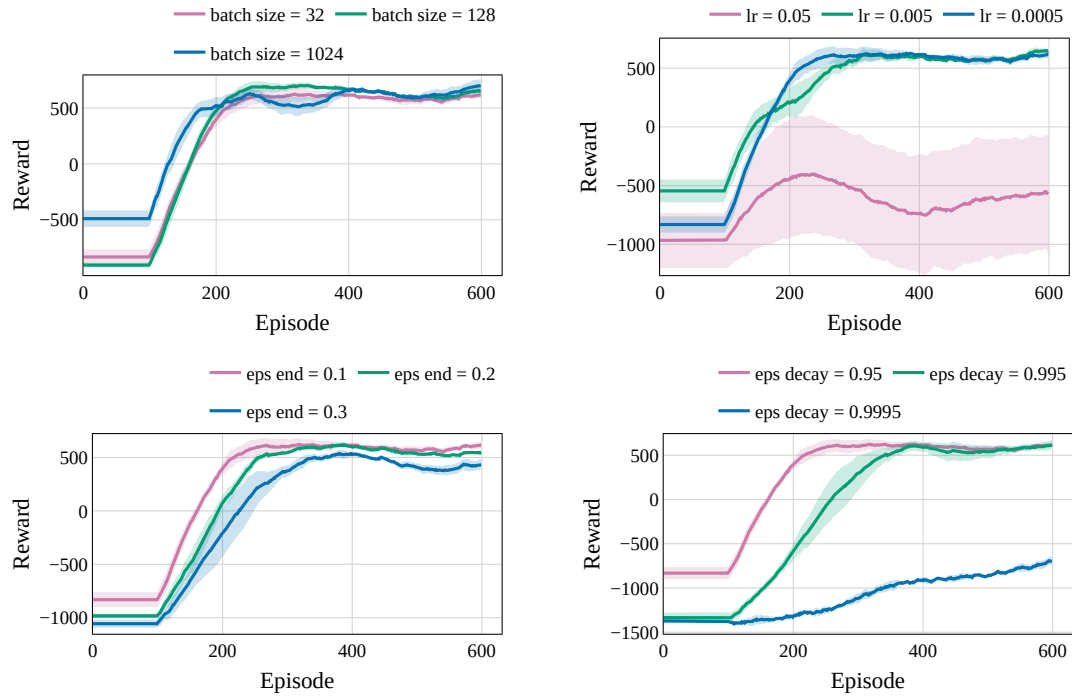


Figure 4.2: Reward per episode for several hyperparameter tuning - Plots obtained by averaging 10 repetitions - The shaded parts corresponds to the standard deviation.

learning rate of 0.05, we can observe that the reward is stabilizing to a certain sub-optimal. The problem is that the learning process is stuck at a local minimum. A high learning rate

can enable one to learn fast but has a high risk of leading to local minima. We observed the experiments with lower learning rates reached a higher reward, meaning they were able to find better local minima.

Secondly, as we decided to use an epsilon greedy policy, we also need to tune the ϵ parameter to have an efficient exploration. We can observe that the final exploration rate has a small influence on the learning speed. Finally, we notice that epsilon decay has a major influence on the learning process. The lower the epsilon decay is, the less the algorithm takes its own decision and relies more on randomness and exploration. Having a low exploration rate can be interesting if we want to maximize exploration but slow the learning process.

In the following table, we introduce the other hyperparameters we choose:

Parameter	Value
learning rate	0.0005
discount factor	0.99
replay buffer size	100000
batch size	32
ϵ_{start}	1
ϵ_{end}	0.1
ϵ_{decay}	0.95
Policy Network Update	4 environment steps
Number of backprop per update	4
Start Learning threshold	10 episodes
Target Network Update	Soft : 0.001

Table 4.1: Chosen hyperparameters

4.1.2 Algorithm and Network

As it was exposed in chapter 2, there are several algorithms and variants of Deep Q-Network. We focus here on the DQN algorithm (algo. 2) and its DL variant: **DDQN** (algo. 3) (fig. 3.6). We also took the time to investigate other network architectures like Dueling DQN (fig. 3.7).

For the classic approach, we can see that the **DDQN** is slightly more stable after 400 episodes than the **DQN**. It has also a better final average reward. For the dueling approach, the observations are similar. Though, the standard deviation of the dueling **DDQN** is less important compared to the simple **DDQN**.

For all following experiments, we will use the CNN-based Dueling **DDQN**.

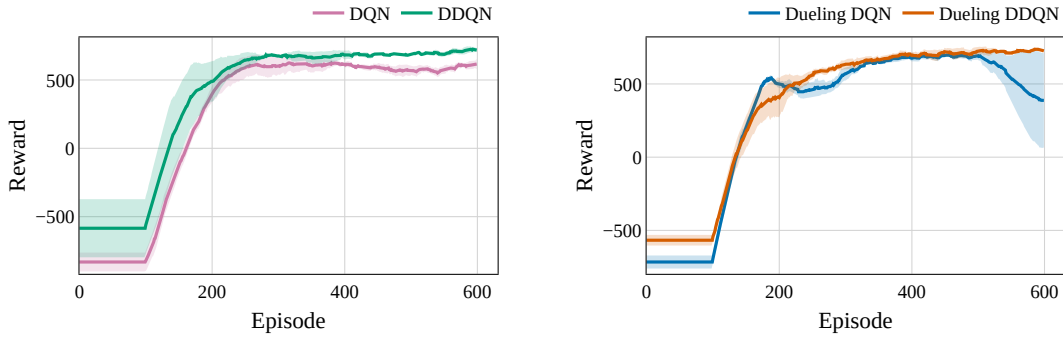


Figure 4.3: Reward per episode for classic approach and dueling approach trained with **DQN** and **DDQN** algorithms - The Dueling DDQN is the approach that has a reduced standard deviation and that reaches the highest reward after 600 episodes - Plots obtained by averaging 10 repetitions - The shaded parts corresponds to the standard deviation.

	DQN	DDQN
Classic	$616,59 \pm 53,10$	$718,65 \pm 21,11$
Dueling	$387,62 \pm 454,88$	$726,08 \pm 3,98$

Table 4.2: Final Averaged Reward obtained over 10 repetitions

	DQN	DDQN
Classic	95.6%	97.2%
Dueling	94.1%	98.6%

Table 4.3: Percentage of the episode where the agent reached the goal obtained over 100 episodes and 10 repetitions

4.1.3 Policy Analysis

We now analyze the trajectories of the trained agent. Over the 10 repetitions that we trained, we kept three of them that perform the best. For the very experiment, the agent is oriented to the North

The trajectories are not always straight and are forming arcs. A possible explanation for this phenomenon is that the agent has a given set of actions that is not continuous. Thus it can only perform certain shapes of trajectory.

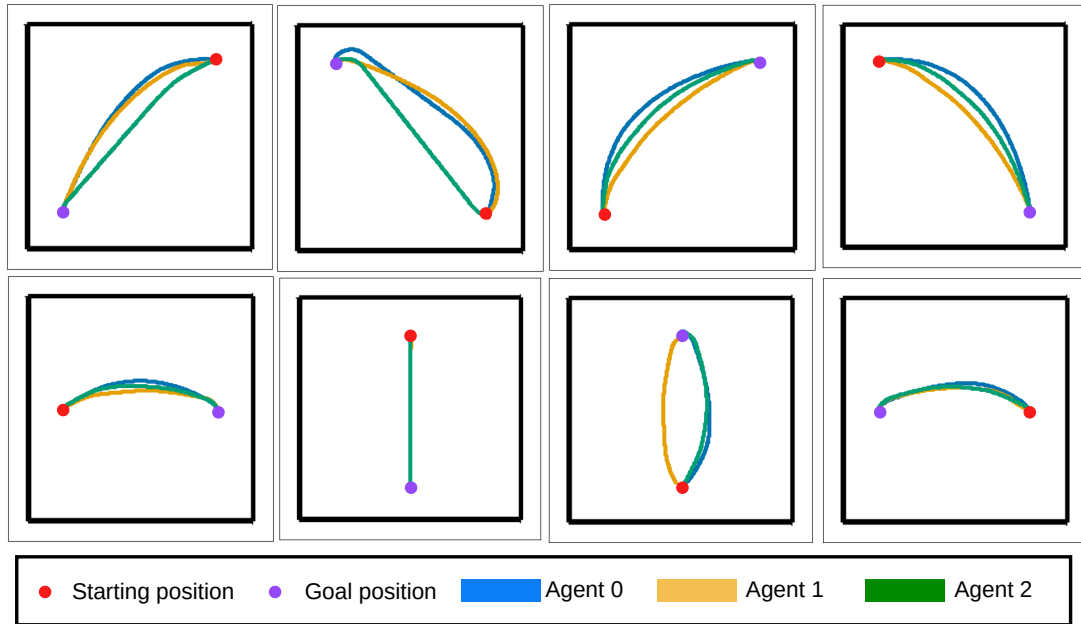


Figure 4.4: Trajectories for end-to-end navigation performed by several RL agents

4.1.4 Comparison with MPC Approach

Finally, we analyze the policy of our trained agent and its performance in comparison with a **MPC** Controller developed the **INRIA** lab research [38]. This control theory algorithm considers a finite Horizon and several spatial constraints. The path planning is done using Fast Marching Method. Every second, the optimal control algorithm uses the path planned and recomputes a local path considering potential objects or entities. We decided to compare the behavior of each approach on certain trajectory scenarios (fig. 4.5).

Approach	RL Agent	MPC Controller
Averaged number of succeed episode	98.6 %	100 %
Averaged Number of steps to reach the goal	147	120
Number of time in No Go Zone	0	0
Averaged time per step in seconds	0.0041	0.043

Table 4.4: Quantitative performance of both approaches over 100 episodes. (An episode is considered as success if the agent reached the goal)

Looking at the trajectories (fig. 4.5) we can see that the **MPC** controller is more

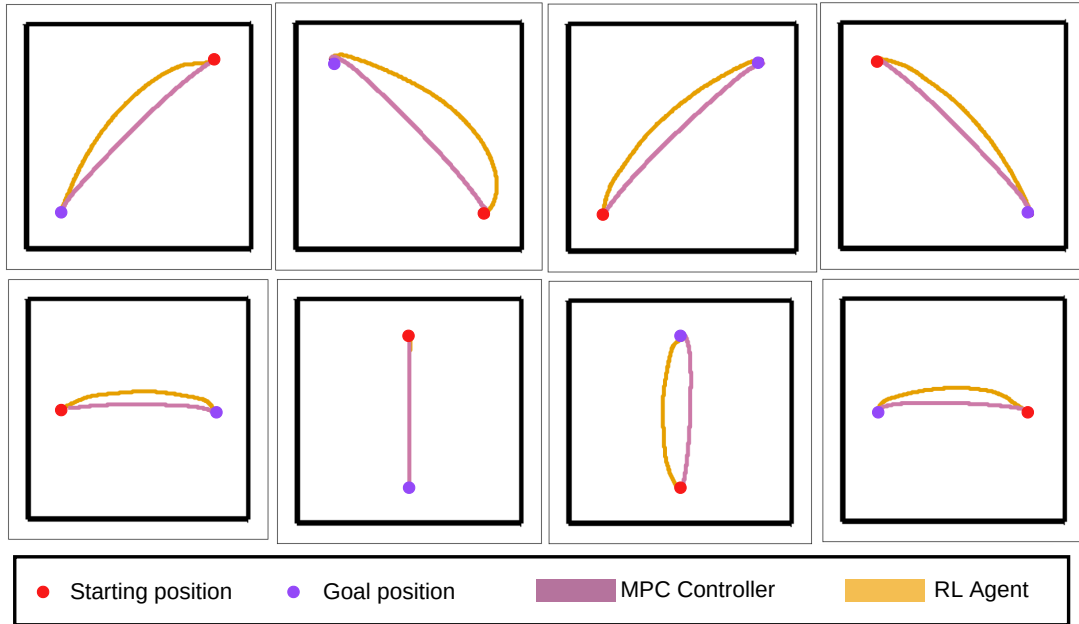


Figure 4.5: Trajectories for end-to-end navigation performed by an RL agent and an MPC Controller

efficient because it goes straight to the goal, whereas the **RL** agent has its "arc-shaped" trajectory. The **MPC** controller has a continuous action space, meaning that it has access to a wide range of angular and linear velocities. There is also a higher rate of success in an episode because sometimes the agent does not reach the goal in the number of maximum steps it has. Nevertheless, our **RL** is 10 times faster than **MPC** in the decision-making process.

4.2 Static Environment Aware Navigation - Results

As we declared before, the state space is still composed of the egocentric local occupancy grid but also of the distance to the goal and the angle to the goal. The neural networks used were the ones presented in fig. 3.6 and fig. 3.7 and the algorithm is the **DDQN**.

The aim is to perform end-to-end navigation while avoiding static objects. The environment where the agent is trained is an empty room with four walls with a table (fig. 4.6). The agent has a random starting and goal position, which is different for every episode. Every experiment was launched at 10 repetitions to obtain intelligible results.

We then averaged each result over these 10 repetitions, which gives a mean (line) and a standard deviation (shaded part) for each data that we looked at. Finally, the maximum number of steps for an episode is 300 and the total number of episodes is 600.

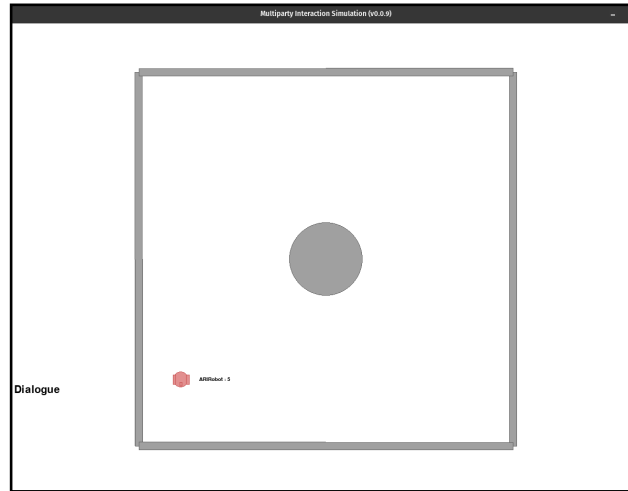


Figure 4.6: Example of simulated room with robot and furniture in the center of the room

4.2.1 Reward Engineering Approach

We are focusing on two types of furniture. The first configuration is where a table is situated at the center of the room during the 1000 episodes of training. A second configuration where a table is randomly situated for each episode. We obtain the following results by looking at the reward per episode.

To know if the agent is too close to furniture, we use an inflated version of the global map. At a given time, when the robot is in that position, the episode ends with a negative reward of -500.

With both configurations, the final average reward is around -100. Seeing the increasing reward, we can observe that the agent understands that he has to avoid the table. Nevertheless, improvements can be made. With that aim, we decided to improve the reward signal.

From now we have only one reference map that defines where the agent must not go and thus when the episode should end. We explored an additional reward function that uses what we call the "warning zone". A "warning zone" will be a zone where the agent can go through but will receive a negative reward.

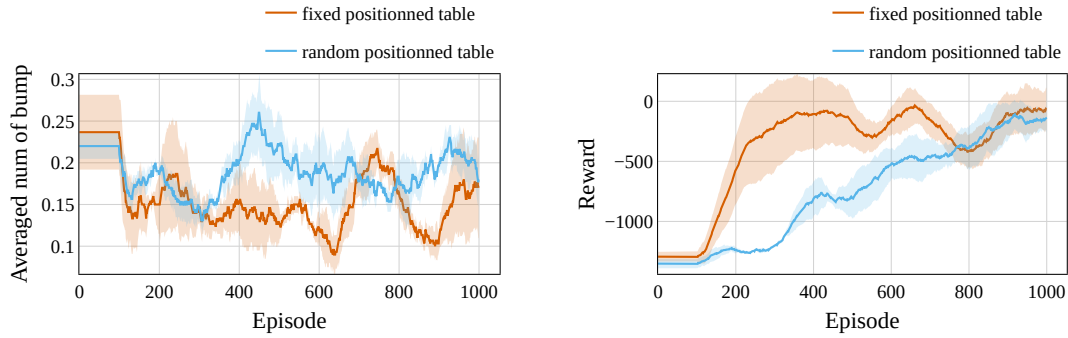


Figure 4.7: Performance of the agent trained in a static environment - Both approach reaches an averaged negative reward and the average number of collision is not decreasing - Plots obtained by averaging 10 repetitions - The shaded parts corresponds to the standard deviation.

$$r_{static} = \begin{cases} -500 & \text{if agent in "no go zone"} \\ -1 & \text{if agent in "warning zone"} \\ 0 & \text{otherwise} \end{cases} \quad (4.1)$$

To detect whether the agent is in a *warning zone* we proceed similarly for *No go zones*. We create another occupancy grid of *warning zone* (fig. 4.8) which defines the positions under which the agent receives the reward r_{static} and the episode continues.

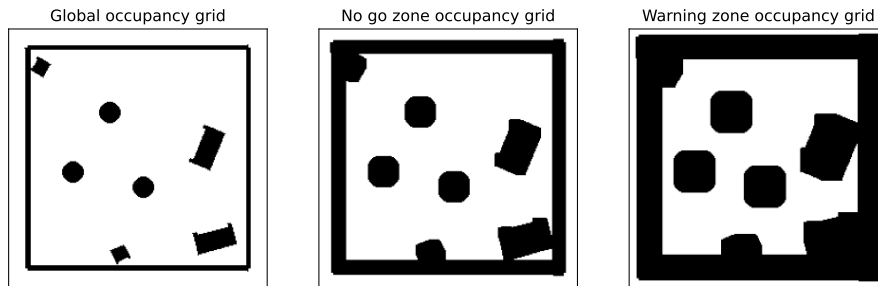


Figure 4.8: Comparison of Global occupancy grid, *No go zone* occupancy grid with *warning zone* occupancy grid

Seeing the results, we observe that this additional reward configuration helps the agent to understand that it has to avoid obstacles. The final average reward is almost the same as before even though we gave the agent a negative reward for being in these forbidden

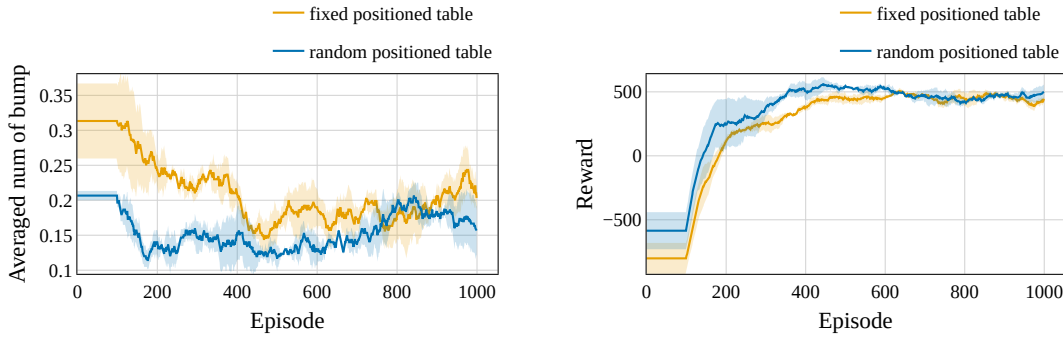


Figure 4.9: Performance of the agent trained in a static environment with the improved reward signal - The agent reaches a reward comparable as the situation when the room is empty but the average number of collisions is more important - Plots obtained by averaging 10 repetitions - The shaded parts corresponds to the standard deviation.

zones. Nevertheless, even after 1000 episodes, the average number of bumps into walls and furniture is still above 15% meaning that our reward function can be improved.

4.2.2 Curriculum Learning Approach

From now, the furniture will be added to the environment room from the beginning of the training procedure. Meaning that the configuration of the room was the same during the entire training time. We decided to try curriculum learning to see if the agent can understand better by dissociating the end-to-end navigation task from the static object avoidance task. Curriculum learning is an approach where the agent learns tasks that have graduate difficulty. In other words, during a certain number of episodes, the agent learns to solve an easy task and after a certain number of episodes, the task gets harder. We choose to consider the end-to-end navigation task easy. So during the first phase (from episode 1 to 500), the agent learns to navigate to the goal. Then we added a table in the room, making the task harder. Then a second table in the upper center part of the room is added after 700 episodes and a third one in the lower center part of the room after 900 episodes. The agent has a task that becomes harder to solve. The reward signal used is the improved reward signal developed in the former sub-part.

We see that the agent performs as expected during the first 500 episodes because the reward increases (fig. 4.10 (a)) to the optimal value and the average number of bumps goes under 5% (fig. 4.10 (c)). Also, as expected, when we add a table in the center, the average number of bumps increases again, meaning that the agent bumps into that table. It results in a decreasing number of steps per episode because each time the agent bumps, the episode ends. Unfortunately, we clearly understand that the agent cannot master

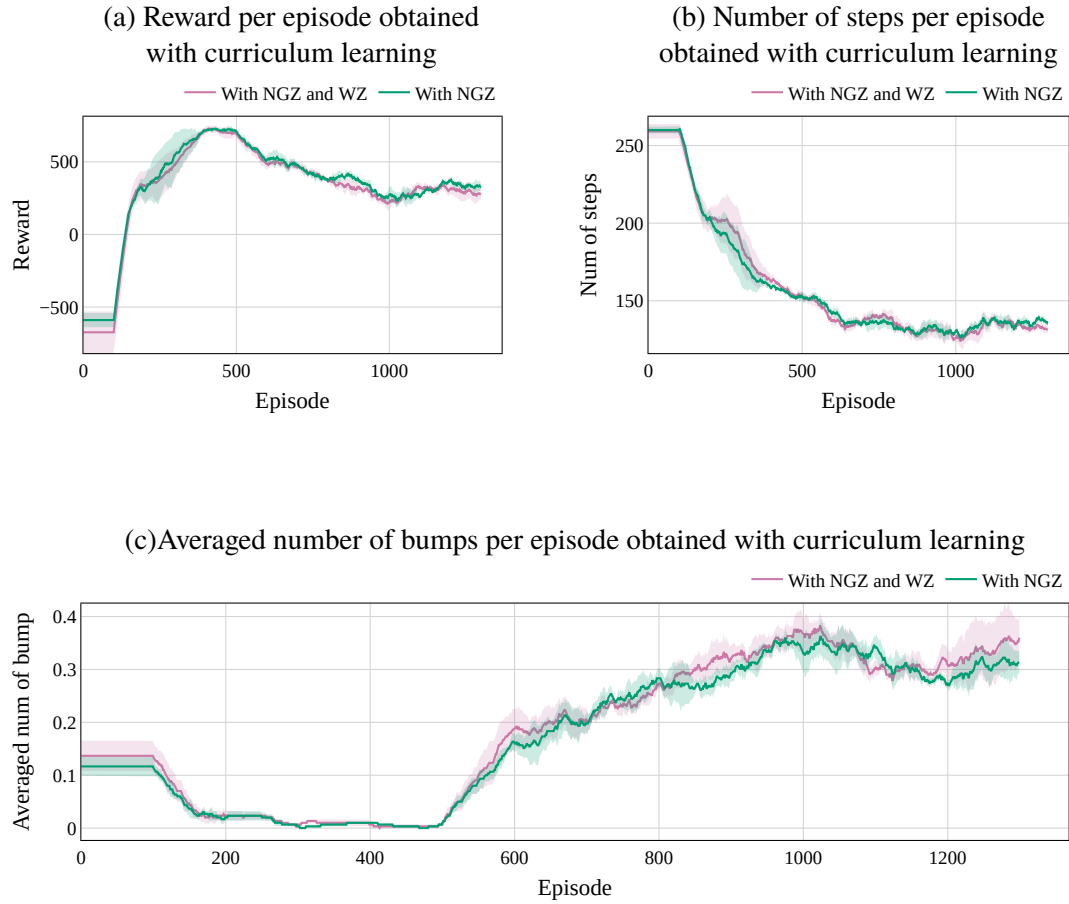


Figure 4.10: Performance of the agent trained in a static environment with curriculum learning for No go zone (NGZ) reward-based and Warning zone (WZ) reward based - Both reward approaches are similar - Plots obtained by averaging 10 repetitions - The shaded parts correspond to the standard deviation.

obstacle avoidance even after 1300 episodes. One explanation could be the fact that the agent requires more episodes. However, considering the possibility that the agent manages to understand the room's configuration, it would not mean that it knows how to avoid obstacles properly. It would have only learned a given configuration and not properly speaking to "avoid obstacles".

4.2.3 Comparison with MPC Approach

Finally, we analyze the policy of our agent trained without curriculum learning and its performance in comparison with a **MPC** Controller developed by other team members

of the **INRIA** lab research. We consider the first condition where a table is on the path between the start position and the goal position. We will then be able to judge how the agent avoided the table. A second condition has a table close to the path between the start and goal position. Here the aim is to see if the warning zone reward has an impact.

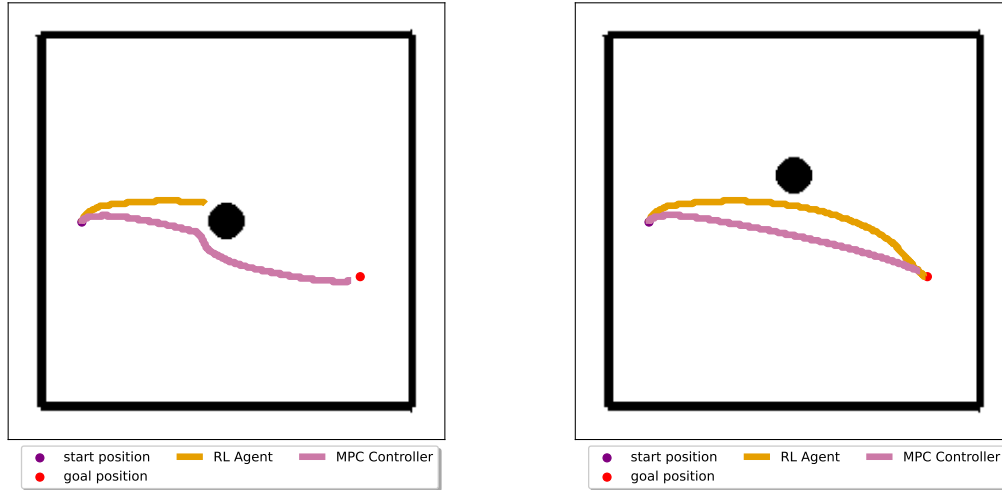


Figure 4.11: Trajectories for end-to-end navigation and obstacle avoidance performed by a **RL** agent and a **MPC** Controller for two configuration of rooms - The **MPC** Controller makes the robot avoid obstacles while the **RL** agent collide into the table and not maneuver away.

When the table is on the way to the goal, the **MPC** Controller makes the robot stop in front of the table, turns on itself and maneuvers away from it to reach the goal. The **RL** agent, go directly to the table without any hesitation. A hypothesis is that the agent's trajectory depends on its action space and that its degrees of freedom suffer from it.

For the second condition, we can observe that the **RL** agent does not maneuver away even if it is in a warning zone. A first explanation could be the fact that the agent did not learn that it was worth it to lose some reward because it could reach the goal. A second explanation follows the former argument that the agent does not maneuver because it does not have enough degrees of freedom.

We can observe with the quantitative results that the **MPC** controller takes more steps to reach the goal than before. It is because it maneuvers around obstacles. Also, the number of times in the warning zone is higher for the **MPC** controller because its own constraint brings it closer to objects. Finally, the fact that the **RL** agent takes fewer steps to reach the goal is due to the fact that it does maneuver and goes to the goal point directly like in the

Approach	RL Agent	MPC Controller
Averaged number of succeed episode	84.2 %	100 %
Averaged Number of steps to reach the goal	190	207
Number of time in No Go Zone	0	0
Number of time in Warning Zone	1.95	11.15
Averaged time per step in seconds	0.0041	0.043

Table 4.5: Quantitative performance of both approaches over 100 episodes. (An episode is considered a succeed if the agent reached the goal)

right figure 4.11.

4.3 Social Environment Aware Navigation - Results

In this final condition, the goal is to perform end-to-end navigation while avoiding humans. The environment where the agent will be trained is going to be a room with four walls and some humans. The trajectory of each human is random (fig. 4.12). The start and goal position of the agent will be random for each episode. The maximum number of steps for a given episode is 300 and the total number of episodes is 600. We evaluated two different approaches based on a modification of the observation space.

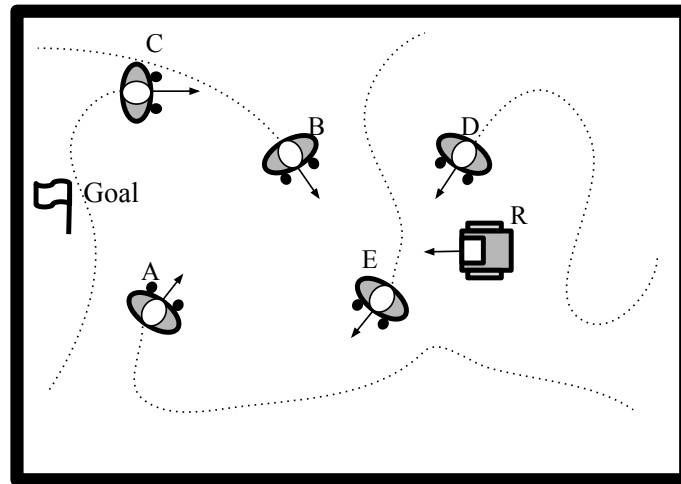


Figure 4.12: Example of trajectory for humans

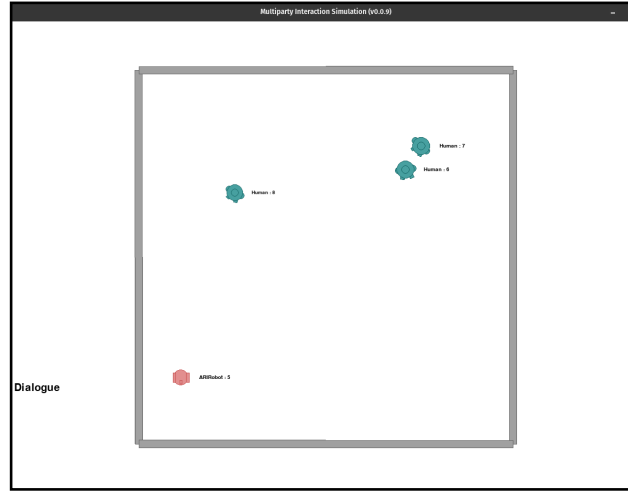


Figure 4.13: Example of simulated room with robot and humans

4.3.1 Layered Input Approach

In this first approach, we consider that the observation space is composed of a multi-layer occupancy grid. There are not only the eventual static objects but there are humans in addition. To make the agent understand its dynamic surroundings, we add three more layers to our egocentric occupancy grid. They contain respectively grids weighted over human positions, orientations, and velocities. When there is a collision, we give the agent a negative reward of -500 and the episode ends. We measure the performance of our agent during its training phase when it has to avoid a given a fixed number of humans. (fig. 4.14).

We note that the reward per episode is increasing and that the final distance between the agent and the goal is decreasing, meaning that the agent learns to go to its goal. Nevertheless, the reward plateau is a different range of values that depends on the number of humans who are in the room. The more humans there are, the more difficult the task becomes and the less the final reward is. Looking at the decreasing number of steps per episode, we understand that it is due to the fact that the agent is colliding with humans. Even if the results seem to mean that the agent does not avoid the humans, we can observe the average number of humans crashed decrease. It makes us hope for the agent's capacity of human avoidance and it means that there is still some progress to achieve.

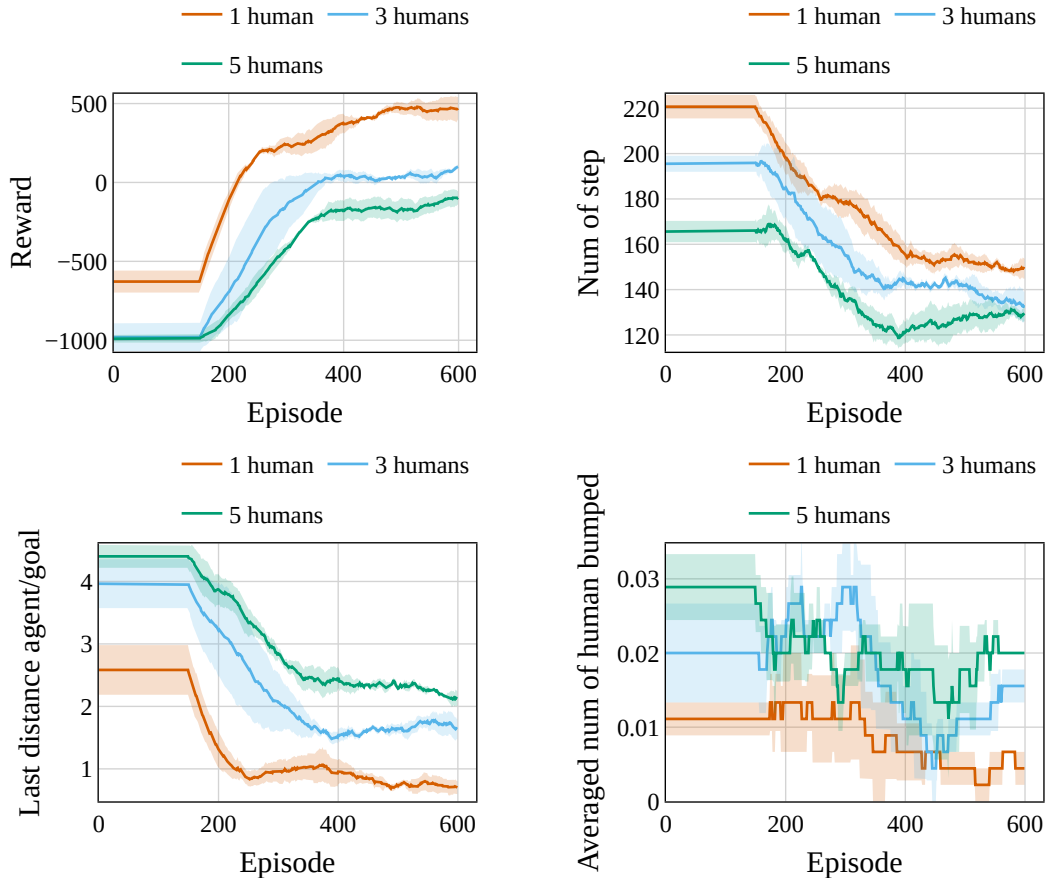


Figure 4.14: Performance of dynamic obstacle avoidance with layered input approach - The agent's performance during training is affected by humans' density - Plots obtained by averaging 10 repetitions - The shaded parts correspond to the standard deviation.

4.3.2 Framed Input Approach

All the information about the surroundings is now contained in a single grid and the agent will have to master the dynamic obstacle avoidance using three successive frames of the dynamic and static occupancy grid. We consider that the observation space is composed of a framed static and dynamic occupancy grid. The reward signal is the same as before and an episode ends if there is any collision.

A similar outcome as with the layered approach can be observed. The framed input approach requires more time to be as efficient as the layered approach.

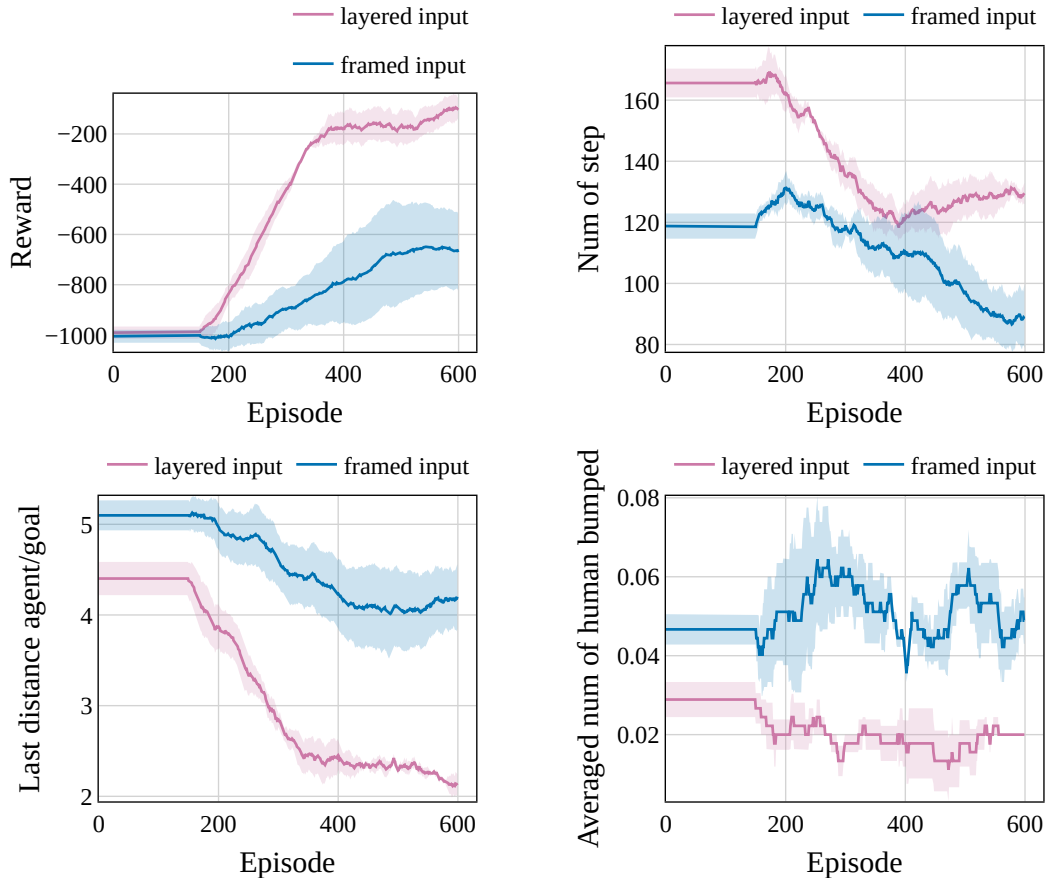


Figure 4.15: Performance of dynamic obstacle avoidance for both layered input and framed input approach - The layered input version reaches a better reward and collides less on average than the framed input version - Plots obtained by averaging 10 repetitions - The shaded parts correspond to the standard deviation.

4.3.3 Comparison with MPC Approach

To finish, we analyze the trajectory of our agent trained with layered input and we compare it to the trajectory monitored by the **MPC** controller. We consider first a situation where the episode ends for both methods when there is a collision between human and robot (left fig. 4.16). The second situation is when we do not stop the episode and the agent continues to go to the goal even though there was a collision (right fig. 4.16). For each situation, there are five humans that are randomly evolving in a given central squared area.

For the first configuration, both methods make robots collide with a human directly. For the second one, we can see that the **RL** agent has its arc-shaped trajectory which helps it to collide with 3 humans. Whereas the **MPC** controller makes the robot go straight to

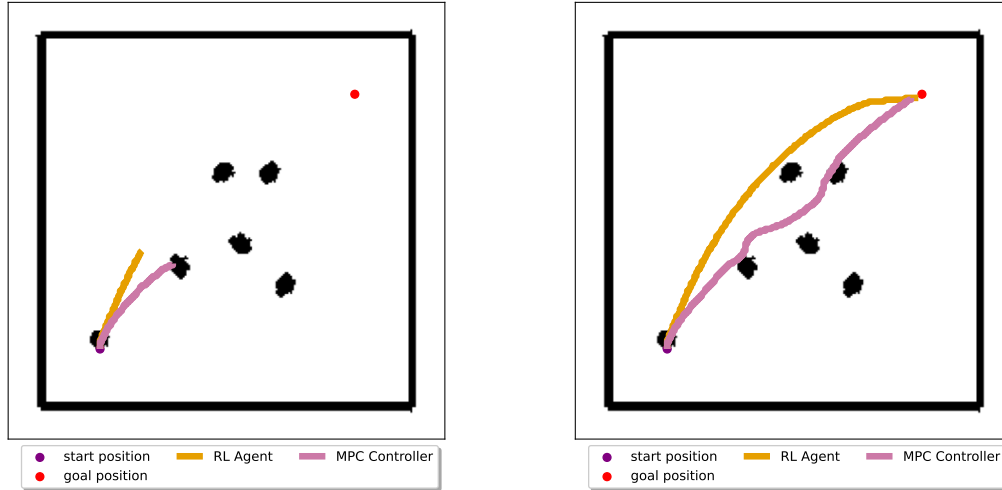


Figure 4.16: Trajectories for end-to-end navigation and human avoidance performed by a **RL** agent and a **MPC** Controller for two configurations of simulation (the background with the humans is just a screenshot to indicate human's area of evolution)

the goal and thus goes through the critical zone where there are humans everywhere. Its S-shaped trajectory shows that it collides with many humans (12 collisions in total). Of course, this situation is special because there is a high concentration of people in a reduced space. Nevertheless, it allows us to see that both techniques are failing to avoid humans. It proves that obstacle avoidance in a crowded area is a challenge that is hard to master for both control theory and **RL**.

Chapter 5

Discussions and Conclusions

This final chapter provides discussions (section 5.1) and conclusions (section 5.2) to this thesis. It finally proposes some potential future research directions (section 5.3).

5.1 Discussions

This section provides a discussion about the different goals defined in the section ??.

The goal of this project is to investigate a **RL** based method in a social environment-aware navigation framework. The first goal was to develop an end-to-end navigation **DRL** algorithm. We implemented a Gym-like environment that used the **MPI** simulation to make it interact with an **RL** algorithm. We compared the impact of several hyperparameters on the learning process, like learning rate and batch size. We also compared the performance of **DQN** and its Dueling and Double variants. Considering that our trained agent is able to reach the goal in 98.6% of cases, we can consider that this goal requirement was reached.

The second goal was to incorporate 2D maps for surrounding awareness. We used the egocentric occupancy grid and proceeded to reward engineering in order to make the agent aware of its surroundings. We observed that even with **CL** it didn't perform that well either. It means that there are still some improvements that can be done. Also, we studied that incorporating additional layers of information about humans can help the agent to avoid them. Still, the resultant agent obtained was not able to avoid humans, revealing that dynamic object avoidance is a hard challenge.

One drawback of this kind of approach is that it demands high computational resources like Graphical Processing Units. To be independent of this necessity, we might use techniques like transfer learning or automatic machine learning.

With this work, we gained insights into how **RL** can solve problems and how to implement

and design an environment that can make an agent master a desired task. We also understood how hard it can be to transform an intuitive behavior into a mathematical equation.

More generally, we hope that this work will give hope for research and inspire others who want to investigate the use of **RL**. We suggest not underestimating the impact of the reward function and not hesitating to be creative when designing it.

5.2 Conclusions

Investigating **RL** based method for navigation in a social environment, this thesis has shown that some challenges like end-to-end navigation can be easy to set up. Whereas other problems, like obstacle avoidance, raise other challenges. This thesis illustrates that using a **RL** approach can help to solve some challenges coming from robotics. It also demonstrates that even if **RL** seems intuitive, there are still things like tuning the reward function that are hard to master. Our model is not effective for obstacle avoidance but it gives an idea of its challenges. It also provides inspiration for using different input types while solving the same task.

5.3 Future work

The first way to continue this work would be to focus on the **RL** improvement. For example, it would be interesting to continue the reward engineering process by giving more clues to the agent on how it can avoid obstacles. Also, as we saw in this work, the action space can have a large impact on the final policy of the agent. One might look for enlarging this action space as much as possible or even make it continuous by using the Deep Deterministic Policy gradient algorithm [39]. With the same idea, a policy-based approach like Actor-Critic Methods [7] or even Proximal Policy Optimization algorithms [40] would be a captivating experience to try.

The second viewpoint to continue this work would be to focus more on the robot. Some previous work tried to work on beeping robots while navigating in a crowded area [41]. Then instead of having only an adaptation coming from the robot, we consider that humans can also adapt their behavior to make the life of the robot easier and safer. A final work would be to realize the implementation of the algorithm on a real robot like a humanoid one.

References

- [1] “irobot website,” <https://www.irobot.com/>, accessed: 2022-12-09. [Page 2.]
- [2] B. Wong, “Color blindness,” *nature methods*, vol. 8, no. 6, p. 441, 2011. [Page 4.]
- [3] V. Dignum, *Responsible artificial intelligence: how to develop and use AI in a responsible way*. Springer, 2019. [Page 4.]
- [4] R. S. Sutton and A. G. Barto, *Reinforcement Learning, second edition: An Introduction*. MIT Press, Nov. 2018. ISBN 978-0-262-35270-3 Google-Books-ID: uWV0DwAAQBAJ. [Page 6.]
- [5] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing Atari with Deep Reinforcement Learning,” Dec. 2013, arXiv:1312.5602 [cs]. [Online]. Available: <http://arxiv.org/abs/1312.5602> [Pages 6, 18, and 22.]
- [6] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” Jul. 2019, arXiv:1509.02971 [cs, stat]. [Online]. Available: <http://arxiv.org/abs/1509.02971> [Page 6.]
- [7] I. Grondman, L. Busoniu, G. A. Lopes, and R. Babuska, “A survey of actor-critic reinforcement learning: Standard and natural policy gradients,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 42, no. 6, pp. 1291–1307, 2012. [Pages 7 and 42.]
- [8] H. Van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016. [Page 9.]
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A. Rusu, J. Veness, M. Bellemare, A. Graves, M. Riedmiller, A. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik,

- I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–33, Feb. 2015. doi: 10.1038/nature14236 [Page 10.]
- [10] P. Mirowski, R. Pascanu, F. Viola, H. Soyer, A. J. Ballard, A. Banino, M. Denil, R. Goroshin, L. Sifre, K. Kavukcuoglu, D. Kumaran, and R. Hadsell, “Learning to Navigate in Complex Environments,” Jan. 2017, arXiv:1611.03673 [cs]. [Online]. Available: <http://arxiv.org/abs/1611.03673> [Page 10.]
- [11] M. Sewak, “Deep q network (dqn), double dqn, and dueling dqn,” in *Deep Reinforcement Learning*. Springer, 2019, pp. 95–108. [Pages 11 and 13.]
- [12] L. C. Baird III, “Advantage updating,” WRIGHT LAB WRIGHT-PATTERSON AFB OH, Tech. Rep., 1993. [Page 11.]
- [13] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, “Rainbow: Combining Improvements in Deep Reinforcement Learning,” Oct. 2017, arXiv:1710.02298 [cs]. [Online]. Available: <http://arxiv.org/abs/1710.02298> [Page 11.]
- [14] F. Hegel, C. Muhl, B. Wrede, M. Hielscher-Fastabend, and G. Sagerer, “Understanding social robots,” in *2009 Second International Conferences on Advances in Computer-Human Interactions*. IEEE, 2009, pp. 169–174. [Page 11.]
- [15] D. Mazzei, F. Chiarello, and G. Fantoni, “Analyzing social robotics research with natural language processing techniques,” *Cognitive Computation*, vol. 13, pp. 308–321, 2021. [Page 11.]
- [16] F. Cavallo, F. Semeraro, L. Fiorini, G. Magyar, P. Sinčák, and P. Dario, “Emotion modelling for social robotics applications: a review,” *Journal of Bionic Engineering*, vol. 15, pp. 185–203, 2018. [Page 11.]
- [17] C. Breazeal, “Social robots for health applications,” in *2011 Annual international conference of the IEEE engineering in medicine and biology society*. IEEE, 2011, pp. 5368–5371. [Page 11.]
- [18] T. Belpaeme, J. Kennedy, A. Ramachandran, B. Scassellati, and F. Tanaka, “Social robots for education: A review,” *Science robotics*, vol. 3, no. 21, p. eaat5954, 2018. [Page 11.]
- [19] C. Breazeal, K. Dautenhahn, and T. Kanda, “Social robotics,” *Springer handbook of robotics*, pp. 1935–1972, 2016. [Page 11.]

- [20] G. Brunner, O. Richter, Y. Wang, and R. Wattenhofer, “Teaching a machine to read maps with deep reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018. [Page 12.]
- [21] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International conference on machine learning*. PMLR, 2016, pp. 1928–1937. [Page 12.]
- [22] N. Akalin and A. Loutfi, “Reinforcement learning approaches in social robotics,” *Sensors*, vol. 21, no. 4, p. 1292, 2021. [Page 12.]
- [23] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba, “Hindsight experience replay,” *Advances in neural information processing systems*, vol. 30, 2017. [Page 12.]
- [24] M. Z. M Liu and W. Zhang, “Goal-conditioned reinforcement learning: Problems and solutions.” Sep. 2022, arXiv:2201.08299. [Online]. Available: <http://arxiv.org/abs/1702.08360> [Page 12.]
- [25] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th annual international conference on machine learning*, 2009, pp. 41–48. [Page 13.]
- [26] J. L. Elman, “Learning and development in neural networks: The importance of starting small,” *Cognition*, vol. 48, no. 1, pp. 71–99, 1993. [Page 13.]
- [27] P. Agarwal, P. de Beaucorps, and R. de Charette, “Goal-constrained sparse reinforcement learning for end-to-end driving,” *arXiv preprint arXiv:2103.09189*, 2021. [Page 13.]
- [28] G. Chen, L. Pan, Y. Chen, P. Xu, Z. Wang, P. Wu, J. Ji, and X. Chen, “Robot navigation with map-based deep reinforcement learning,” *arXiv preprint arXiv:2002.04349*, 2020. [Page 13.]
- [29] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” *arXiv preprint arXiv:1511.05952*, 2015. [Page 13.]
- [30] G. Chen, S. Yao, J. Ma, L. Pan, Y. Chen, P. Xu, J. Ji, and X. Chen, “Distributed non-communicating multi-robot collision avoidance via map-based deep reinforcement learning,” *Sensors*, vol. 20, no. 17, p. 4836, 2020. [Page 13.]

- [31] X.-T. Truong and T.-D. Ngo, ““to approach humans?”: A unified framework for approaching pose prediction and socially aware robot navigation,” *IEEE Transactions on Cognitive and Developmental Systems*, vol. 10, no. 3, pp. 557–572, 2018. doi: 10.1109/TCDS.2017.2751963 [Page 13.]
- [32] S. Yao, G. Chen, Q. Qiu, J. Ma, X. Chen, and J. Ji, “Crowd-aware robot navigation for pedestrians with multiple collision avoidance strategies via map-based deep reinforcement learning,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 8144–8150. [Page 13.]
- [33] Y. F. Chen, M. Everett, M. Liu, and J. P. How, “Socially aware motion planning with deep reinforcement learning,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 1343–1350. [Page 13.]
- [34] “Pygame website,” <https://www.pygame.org/>, accessed: 2022-12-01. [Page 15.]
- [35] “Box2d official website,” <https://box2d.org>, accessed: 2022-02-01. [Page 16.]
- [36] “Openai gym library website,” <https://www.gymnasium.dev/>, accessed: 2023-01-20. [Page 16.]
- [37] G. Chen, L. Pan, Y. Chen, P. Xu, Z. Wang, P. Wu, J. Ji, and X. Chen, “Robot Navigation with Map-Based Deep Reinforcement Learning,” in *2020 IEEE International Conference on Networking, Sensing and Control (ICNSC)*. Nanjing, China: IEEE, Oct. 2020. doi: 10.1109/ICNSC48988.2020.9238090. ISBN 978-1-72816-853-1 978-1-72816-855-5 pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/9238090/> [Page 22.]
- [38] “Spring project. d6.1: Neural network architecture specification and design.” https://spring-h2020.eu/wp-content/uploads/2021/03/SPRING_D6.1_Neural-network-architecture-specification-and-design_VFinal_24.02.2021.pdf, accessed: 2023-01-07. [Page 29.]
- [39] J. C. Jesus, J. A. Bottega, M. A. Cuadros, and D. F. Gamarra, “Deep deterministic policy gradient for navigation of mobile robots in simulated environments,” in *2019 19th International Conference on Advanced Robotics (ICAR)*. IEEE, 2019, pp. 362–367. [Page 42.]
- [40] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. [Page 42.]

- [41] Y. Nakamura, T. Mori, M.-a. Sato, and S. Ishii, “Reinforcement learning for a biped robot based on a cpg-actor-critic method,” *Neural networks*, vol. 20, no. 6, pp. 723–735, 2007. [Page 42.]

Appendix A

Additional Pseudo-Code

Algorithm 2 Deep Q-Learning with Experience Replay

 Initialize replay memory D to capacity N

 Initialize action-value function Q_θ with random weight

for *each episode* **do**

 Observe s_0

 Initialize environment step $t \leftarrow 0$

 Initialize update step $j \leftarrow 0$
for *each environment step* **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_b Q_\theta(s_t, b)$

 Execute action a_t and observe reward r_t and next state s_{t+1}

 Store a transition (s_t, a_t, r_t, s_{t+1}) in D
for *each update step* **do**

 Sample a random batch of transitions from $D : (s_j, a_j, r_j, s_{j+1})$

 Compute target value $y_j = r_j + \gamma \max_b Q_\theta(s_j, b)$ if s_j is not a terminal state

 otherwise $y_j = r_j$

 Perform a gradient descent step on $(y_j - Q_\theta(s_j, a_j))^2$

Algorithm 3 Deep Q-Learning with Experience Replay and Target Network

Initialize replay memory D to capacity N Initialize action-value functions Q_θ (Policy) and Q_ϕ (Target) with random weights**for each episode do** Observe s_0 Initialize environment step $t \leftarrow 0$ Initialize update step $j \leftarrow 0$ **for each environment step do** With probability ϵ select a random action a_t otherwise select $a_t = \operatorname{argmax}_b Q_\phi(s_t, b)$ Execute action a_t and observe reward r_t and next state s_{t+1} Store a transition (s_t, a_t, r_t, s_{t+1}) in D **for each update step do** Sample a random batch of transitions from $D : (s_j, a_j, r_j, s_{j+1})$ Compute target value $y_j = r_j + \gamma Q_\theta(s_j, \operatorname{argmax}_b (Q_\phi(s_{j+1}, b)))$ if s_j is not a terminal state otherwise $y_j = r_j$ Perform a gradient descent step on $(y_j - Q_\theta(s_j, a_j))^2$ Update the target network parameters $\phi = \theta.\tau + \phi.(1 - \tau)$

